Applying Machine Learning in Self-adaptive Systems: A Systematic Literature Review

OMID GHEIBI, Katholieke Universiteit Leuven
DANNY WEYNS, Katholieke Universiteit Leuven, Linnaeus University
FEDERICO QUIN, Katholieke Universiteit Leuven

Recently, we have been witnessing a rapid increase in the use of machine learning techniques in self-adaptive systems. Machine learning has been used for a variety of reasons, ranging from learning a model of the environment of a system during operation to filtering large sets of possible configurations before analyzing them. While a body of work on the use of machine learning in self-adaptive systems exists, there is currently no systematic overview of this area. Such an overview is important for researchers to understand the state of the art and direct future research efforts. This article reports the results of a systematic literature review that aims at providing such an overview. We focus on self-adaptive systems that are based on a traditional Monitor-Analyze-Plan-Execute (MAPE)-based feedback loop. The research questions are centered on the problems that motivate the use of machine learning in self-adaptive systems, the key engineering aspects of learning in self-adaptation, and open challenges in this area. The search resulted in 6,709 papers, of which 109 were retained for data collection. Analysis of the collected data shows that machine learning is mostly used for updating adaptation rules and policies to improve system qualities, and managing resources to better balance qualities and resources. These problems are primarily solved using supervised and interactive learning with classification, regression, and reinforcement learning as the dominant methods. Surprisingly, unsupervised learning that naturally fits automation is only applied in a small number of studies. Key open challenges in this area include the performance of learning, managing the effects of learning, and dealing with more complex types of goals. From the insights derived from this systematic literature review, we outline an initial design process for applying machine learning in self-adaptive systems that are based on MAPE feedback loops.

CCS Concepts: • **Software and its engineering**; • **Computing methodologies** → **Machine learning**; • **General and reference** → **Surveys and overviews**;

Additional Key Words and Phrases: Self-adaptation, feedback loops, MAPE-K

ACM Reference format:
Omid Gheibi, Danny Weyns, and Federico Quin. 2021. Applying Machine Learning in Self-adaptive Systems: A Systematic Literature Review. *ACM Trans. Auton. Adapt. Syst.* 15, 3, Article 9 (August 2021), 37 pages.
https://doi.org/10.1145/3469440

Authors' addresses: O. Gheibi and F. Quin, Katholieke Universiteit Leuven, Celestijnenlaan 200A, Leuven, Belgium, 3000 Leuven, Belgium; emails: {omid.gheibi, federico.quin}@kuleuven.be; D. Weyns, Katholieke Universiteit Leuven, Celestijnenlaan 200A, 3000 Leuven, Belgium, and Linnaeus University, 351 95 Vaxjo, Sweden; email: danny.weyns@kuleuven.be.

1556-4665/2021/08-ART9 $15.00
https://doi.org/10.1145/3469440

1 INTRODUCTION

The ever-growing complexity of software systems that need to maintain their goals 24/7 while operating under uncertainty motivates the need to equip systems with mechanisms to handle change during operation. An example is a cloud service that needs to satisfy user performance and minimize operational costs, while operating under dynamically changing workloads. This service may be enhanced with an elasticity module that adjusts resources with changing workload.

A common approach to handle change is the use of "internal mechanisms," such as exceptions (as a feature of a programming language) and fault-tolerant protocols. The application of such mechanisms is often domain-specific and tightly bound to the code. This makes it costly to build, modify, and reuse solutions [35]. In contrast, change can be handled using "external mechanisms" that are based on the concept of a feedback loop. An important paradigm in this context is self-organization, where relatively simple elements apply local rules to adapt their interactions with other elements in response to changing conditions to cooperatively realize the system goals [39, 69]. Another important paradigm is control-based adaptation that relies on the mathematical basis of control theory for designing feedback loop systems and analyzing and guaranteeing key properties [1, 83].

In this article, we focus on architecture-based adaptation, which is an extensively studied and applied approach to handle change [12, 35, 50, 76, 96, 102]. Architecture-based adaptation relies on a feedback loop that monitors the system and its context and adapts the system to ensure its goals, or degrade gracefully if necessary. Pivotal in tackling this task is the use of runtime models [35, 76] that enable the system to reason about (system-wide) change and make adaptation decisions. The feedback loop localizes the adaptation concerns in separable system elements that can be analyzed, modified, and reused across systems. Examples of practical applications of architecture-based adaptation are management of renewable energy production plants [15], information systems for public administration [16], and automation of the management of Internet of Things applications [100].

Realizing feedback loops for architecture-based adaptation is, in general, not a trivial task. Over the past years, several techniques have been investigated to support the design and operation of self-adaptation. One of these techniques is search-based software engineering. For instance, Reference [13] argues for the use of evolutionary computation to generate and analyze models of dynamically adaptive systems to deal with uncertainties both at development time and runtime. Our focus in this article is on another prominent line of research that applies machine learning techniques in the design and operation of self-adaptation. We highlight a few of the incentives to apply machine learning techniques to architecture-based self-adaptive systems. Online verification of rigorously specified runtime models enables providing guarantees about the adaptation decisions. However, formal verification of runtime models for all possible adaptation options during operation can be time consuming. We may then use a learning mechanism to filter the configurations before starting analysis. Another challenging aspect is the design of runtime models of complex software systems. These models may become particularly complicated up to the level that it may be infeasible to design the models if the structure of the system or its context is not known beforehand. Hence, the models need to be derived during operation, for which we may use learning techniques.

Currently, we have a sizeable body of work in this area. Yet, even though the amount of research that has been conducted on this topic is quite substantial, there is no clear view on the state of the art. Such an overview is important for researchers in this area as it will document the current body of knowledge and clarify open challenges. To tackle this problem, we performed a systematic literature review [49]. The goal of this study is to provide a systematic overview of the state of the art on the application of machine learning methods in self-adaptation. The survey is centered on (i) the problems that motivate the use of machine learning in self-adaptive systems, (ii) key engineering aspects of learning applied in self-adaptation, and (iii) open challenges.

The remainder of this article is structured as follows. Section 2 starts with explaining background and outlining the focus of the literature review. In Section 3, we position this review in the landscape of related review work. Section 4 then summarizes the protocol of the study that includes the research questions, the search string, inclusion and exclusion criteria, the data items that we extracted from the papers, and the methods we used for the analysis. Section 5 reports the results from the analysis of the collected data, providing answers to the research questions. In Section 6, from the main findings of the literature review, we present an initial design process for applying machine learning in self-adaptive systems, we present opportunities for further research, and we discuss threats to validity. Finally, we wrap up and conclude the article in Section 7.

2 BACKGROUND AND REVIEW FOCUS

In this section, we briefly introduce self-adaptive software with a **Monitor-Analyze-Plan-Execute (MAPE)**-based feedback loop, and we summarize the essential dimensions of machine learning methods. Then, we clarify the distinction between an adaptation problem and a learning problem in the context of this study, and we highlight the importance of the use of machine learning in self-adaptive systems.

2.1 MAPE-based Self-adaptation

This study focuses on self-adaptive systems based on architecture-based adaptation [35, 54, 102]. Such a self-adaptive system comprises two parts: a *managed system* that is controllable and subject to adaptation and the *managing system* that performs the adaptations of the managed system. The managed system operates in an *environment* that is non-controllable. The managing system realizes a feedback loop that comprises four essential functions: Monitor-Analyze-Plan-Execute that share Knowledge, often referred to as MAPE-K or MAPE [50]. The monitor tracks the managed system and the environment in which the system operates and updates the knowledge. The analyzer uses the up-to-date knowledge to evaluate the need for adaptation, possibly exploiting rigorous analysis techniques [6, 9, 41] or simulations of runtime models [99]. Such analysis may apply rigorous methods to provide guarantees for the If adaptation is required, it analyzes alternative configurations of the managed system. We refer to these alternative configurations as the adaptation options. The planner then selects the best option based on the adaptation goals and generates a plan to adapt the system from its current configuration to the new configuration. Finally, the executor executes the adaptation actions of the plan. It is important to highlight that MAPE provides a reference model that describes a managing system's essential functions and the interactions between them. A concrete architecture maps the functions to corresponding components, which can be a one-to-one mapping or any other mapping, such as a mapping of the analysis and planning functions to one integrated decision-making component.

In this literature review, we consider studies that are based on the MAPE reference model that maps the MAPE functions (or some of them) to a specific component-based architecture.

2.2 Machine Learning

T. Mitchell defines machine learning as follows: "a computer program is said to learn from experience E concerning some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E" [65]. For example, consider a self-adaptive sensor network that needs to keep packet loss and latency under given thresholds. The training experience (E) from which we learn could be the results of the analysis of adaptation options. The task (T) could be a classification of the adaptation options in two classes: those that are predicted to comply with the goals (and should be analyzed) and those that are predicted not to comply (and should not be analyzed). The performance measure (P) to perform this task could be the comparison of

predicted values of packet loss and latency with the threshold values of the respective adaptation goals. In this example, learning (classification) supports the analysis stage of the feedback loop by reducing a large number of adaptation options, aiming to improve the efficiency of analysis.

In the field of machine learning, a distinction can be made in four dimensions [4, 80]:

- *Unsupervised vs. Supervised vs. Interactive*: An unsupervised learner aims at finding previously unknown patterns in data sets without preexisting labels. One of the main methods used in unsupervised learning is cluster analysis. Cluster analysis identifies commonalities in the data and reacts based on the presence or absence of such commonalities in new data. A supervised learner learns a function that maps an input to an output based on example input-output pairs. The function is inferred from labeled data and can then be used to map new data. An interactive learner collects the input-output pairs by interaction with the environment. The learner uses a learning model to make predictions that are then used to perform actions in the environment. The environment then provides feedback on the actions that the learner incorporates in its learning model improving the learning process; a classic example is reinforcement learning. We refer to the basic dimension that distinguishes unsupervised, supervised, and interactive learning as *learning type*.
- *Active vs. Passive*: In active learning a learning algorithm interactively queries some information source in the environment (e.g., a user or teacher) to obtain the desired outputs at new data points. These outputs in turn are used to affect the environment. A passive learner only perceives the information from the environment without affecting it.
- *Adversarial vs. Non Adversarial*: Adversarial learning attempts to fool models through malicious input. This technique can be applied to attack standard learning models. An example is an attack in spam filtering, where spam messages are obfuscated through the manipulation of the text. Non adversarial learning has no concept of malicious input.
- *Online vs. Batch Protocol*: In online learning data becomes available in a sequential order and is used to update the learning model for future data at each step. Batch learning, however, generates the learning model by learning on the entire training data set at once.

In the literature review, we consider the four dimensions of machine learning methods.

2.3 Adaptation Problem versus Learning Problem

As explained above, in this survey, we target software systems that comprise two parts: a managed system that is adapted by a managing system, as illustrated in Figure 1. The managing system is based on a MAPE-K feedback loop that solves an *adaptation problem*. An adaptation problem relates to one or more concerns of a managed system that typically pertain to quality properties. The MAPE-K feedback loop is supported by a *machine learner* that solves a particular *learning problem*. Hence, in this study, we look at learning problems that are part of adaptation problems.

Consider the example we used in the introduction of a cloud service as a managed system. The adaptation problem is to ensure user performance while minimizing operational cost for the owner under changing workloads. To that end, the cloud service is extended with an elasticity module. This module realizes a feedback loop that dynamically adjusts the resources of the cloud service based on changing workloads. A learning problem in this context may be the prediction of the workload of the cloud service. Such learner would support the monitor and analyzer of the feedback loop of the elasticity module. Hence, the elasticity of the cloud service is realized by the collaboration of the MAPE-K feedback loop and the machine learner that together form the managing system.

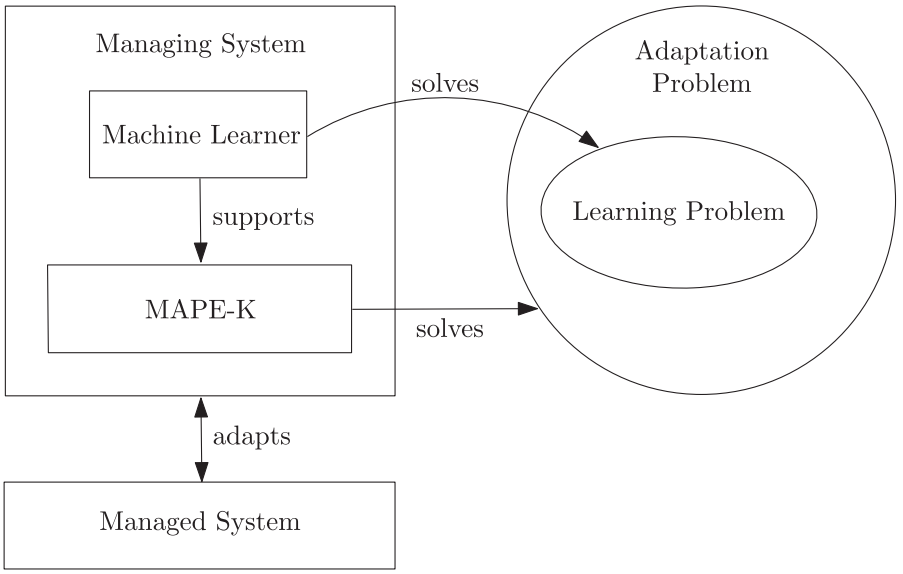

Fig. 1. Relation of learning problem and adaptation problem with the components of the managing system.

2.4 Importance of Machine Learning in Self-Adaptive Systems

A recent book structures the field of self-adaptation in seven "waves" that highlight the important research areas of the past two decades that have contributed to the current body of work [97]. The seventh wave focuses on machine learning techniques as a means to enhance the realization of a self-adaptive software system. The book argues that machine learning can be used to support different activities of the MAPE workflow of self-adaptive systems and highlights three characteristic use cases. The first use case enhances the monitor function of a self-adaptive system with a Bayesian estimator that keeps a runtime model up to date. A concrete example is described in Reference [27]. This simple use case underpins the power of machine learning when dealing with parametric uncertainties represented in runtime models. The second use case enhances the analyzer function with a classifier that enables large sets of adaptation options to be reduced at runtime, improving the efficiency of the analysis phase of self-adaptation. A concrete example is described in Reference [75]. This use case shows how learning can help to deal with the complexity that comes with the increasing scale of self-adaptive systems. Finally, the third use case enhances various feedback loop functions with a learning strategy that combines fuzzy control and fuzzy Q-learning to adjust and improve auto-scaling rules of a cloud infrastructure at runtime. An example is described in Reference [44]. This use case shows how machine learning can help to support decision making in self-adaptive systems that are subject to complex types of uncertainties.

These examples underpin the importance of machine learning techniques in self-adaptive systems. The aim of this article is to study the use of machine learning in self-adaptation in a systematic way to document the current body of knowledge in this area and identify open challenges.

3 RELATED REVIEWS

A number of reviews related to this study have been published, but they differ in the methodology used, the domain studied, or the types of feedback loops considered. We highlight key aspects of these studies and position our work to these reviews. Table 1 summarizes the related work.

Table 1. Summary of Related Reviews

Related review	Method	Domain	Type feedback loop
Klaine et al. [52]	Literature review	Self-organizing networks	Control-theoretic
D'Angelo et al. [17]	SLR	Collective self-adaptive systems	Decentralized
Gambi et al. [34]	Not specified	Cloud	Not constrained
Lorido-Botran et al. [61]	Not specified	Cloud	Not constrained
Masdari et al. [64]	Not specified	Cloud	Systems in general
Cui et al. [14]	Literature review	IoT	Systems in general
Saputri and Lee [78]	SLR	Self-adaptive systems	Systems in general

SLR refers to Systematic Literature Review.

Klaine et al. [52] performed a review on machine learning in self-organizing networks. The authors classified papers based on their learning solution and self-organizing use-case. Moreover, they provided a general guideline to deciding which machine learning algorithm is proper for which use-case in self-organizing networks. That work has three main differences compared to our systematic literature review: (1) methodology: their work is a basic review without any specific query, data extraction and analysis phase, in contrast to our study, which is a systematic literature review; (2) domain: their work focused on a specific domain, i.e., self-organizing networks, in contrast to our work that does not put any constraint on the application domain in self-adaptive systems; (3) managing system: their work mainly focused on control-theoretic feedback loops, in contrast to our work that focuses on self-adaptive systems with MAPE-based feedback loops.

D'Angelo et al. [17] presented a three-dimensional framework to categorize and compare work on learning capabilities in collective self-adaptive systems. This framework approached the applied learning methods from three dimensions: autonomy, knowledge access, and behavior. Although that work is analogous with our work in terms of using machine learning in self-adaptive systems, the main differences are: (1) research methodology: they followed the principles of a literature review but with a more specific focus than our study; (2) domain: they focused on decision-making realized by so called collective self-adaptive systems, while our work goes beyond decentralized decision-making and considers all types of MAPE-based self-adaptive systems that are selected via inclusion/exclusion criteria; (3) feedback loops: they focused on decentralized self-adaptive systems or multi-agent systems, while our work considers all types of MAPE-based self-adaptive systems.

Gambi et al. [34] studied self-adaptive controllers applied in the Cloud. The authors focus on two main dimensions: adaptability (flexibility and scope of the adaptation of the system) and reliability (accuracy of the assurances for reliability). Based on these dimensions, the authors categorized existing control methods in three groups: (1) controllers that prefer adaptability over reliability; (2) controllers that prefer reliability over adaptability; (3) controllers that balance between the two. The main differences with our work are: (1) research methodology: their work is a basic literature study, while our study follows the guidelines of a systematic literature review; (2) domain: their work is focused on the Cloud domain, while we consider all application domains. Moreover, their work focused on specific concerns in Cloud without a particular focus on learning methods. In contrast, our work aims at identifying existing issues in a variety of domains that have been tackled by machine learning; (3) managing system: the proposed work has no particular constraints in terms of control approaches used, while our work is scoped on MAPE-based feedback loop systems.

Lorido-Botran et al. [61] classified techniques used for auto-scaling of Cloud environments. The authors identified five groups of techniques: threshold-based rules, reinforcement learning, queuing theory, control theory, and time series analysis. The main differences with our work are: (1) research methodology: the authors do not report any specific research methodology they applied in their work. In contrast, we applied a systematic literature review; (2) domain: their

work focused on a specific concern (auto-scaling) within a particular application domain (Cloud). In contrast, problems and domains are open in our study, and the focus is on the use of learning methods to support self-adaptation; (3) feedback loops: there is no limitation on controlling approaches in their work, opposite to our criteria to focus on MAPE-based feedback loops.

Another survey studied the classification of workload prediction methods in cloud computing [64]. The motivation for this review is the crucial role of the workload prediction in auto-scaling and resource management of clouds. As a result, the authors identified 11 classes of prediction methods, including regression-based, classifier-based, stochastic-based, and wavelet-based methods. The main differences with our work are: (1) research methodology: in contrast to our systematic literature review, there is no specific research methodology applied in the presented work; (2) domain: the presented work focuses on specific methods to solve specific problems in the Cloud, whereas we target a broad range of problems and application domains; (3) feedback loops: they focus on general prediction methods without assuming any particular system structure, while our work is scoped on utilizing machine learning techniques in architecture-based self-adaptive systems.

Cui et al. [14] performed a survey on the application of machine learning in the IoT domain, e.g., for security, traffic profiling, and device identification in IoT. Two main classes of application domains that they identified are personal health and industrial applications. However, our work is different from their work in the following terms: (1) research methodology: their work is a basic literature study, while we studied the literature using a systematic method; (2) domain: their work focused on the domain of IoT, while we did not put any constraints in our study on application domains; (3) feedback loops: their work is not limited to self-adaptive systems, although the work is helpful for self-adaptation. Hence, in their work, the nature of the managing system does not matter, in contrast to our work that targets MAPE-based feedback loop systems.

Finally, Saputri and Lee [78] performed a literature review that aimed at helping software engineers in proper selection of machine learning techniques based on adaptation concerns at hand. Based on the results, the authors proposed an initial taxonomy for choosing machine learning techniques in self-adaptation. The results of this study are difficult to interpret as the authors mix learning types, learning tasks, and learning methods. Similarly, the authors used a concept called "concern objects for adaptation in self-adaptive systems" that "refers to the concerns in the adaptation that are addressed using machine learning approaches." These "concerns objects" are architecture, behavior, framework, model, and verification. It is not clear why these concepts were chosen and neither is it clear how a classification can be done based on these concepts as they have obvious overlaps. Some of the other differences compared to our study are:(1) methodology: the authors performed a systematic literature review, yet, it is remarkable that the automatic search resulted in only 315 papers (78 selected) compared to over 6,700 papers in our study (109 selected); (2) domain: the paper states that the focus of their work is self-adaptation, yet, a variety of papers are included on agents, self-organization, and general software systems; (3) feedback loops: the paper states that it focuses on self-adaptation, yet, the authors report that 55 of the 87 selected papers do not have any concept of MAPE; the term feedback loop is not mentioned in the paper.

4 SUMMARY PROTOCOL

This study used the methodology of a systematic literature review as described in Reference [49]. The methodology defines the way in which a literature review should be performed so that the relevant papers are properly identified, evaluated, and interpreted. A systematic literature review is composed of three stages: planning, execution, and reporting. During the planning stage, a protocol is defined for the study. This protocol includes the research questions of the study, the sources to search for papers, the search string to collect papers, inclusion and exclusion criteria

to select relevant papers, and the data items that need to be collected from the selected papers to answer the research questions. In the execution phase the search string is applied, papers are collected, and the data is extracted. In the reporting phase the collected data is analyzed and interpreted, the research questions are answered, useful insights are documented, and potential threats to the validity of the study are discussed.

We conducted the systematic literature review with three researchers that jointly developed the protocol. One researcher performed the automatic search, resulting in 6,709 papers. To avoid bias when selecting papers, i.e., the primary studies, we applied the inclusion and exclusion criteria as follows. For a first batch of 196 randomly identified papers two researchers selected papers. The results were then compared and in case of differences, the two researchers resolved any conflicts. If no consensus could be reached, then the third researcher was involved to make a decision in consensus. We repeated this process until the two reviewers selected the same papers. Concretely, we used three more rounds, where the researchers applied the inclusion/exclusion criteria, respectively, to 111, 66, and 47 randomly selected papers. The number of conflicts decreased to zero in the final round. One researcher then applied the inclusion/exclusion criteria to the remaining papers. We used a similar process for the extraction of the data. We applied two rounds of data extraction, in each round two researchers extracted data of five papers. One researcher then extracted the data of the remaining papers and a sample of five other papers were crosschecked by the two other researchers. Finally, the analysis and reporting was jointly done by the three researchers (we explain the process we used in Section 4.5).

We now briefly explain the main parts of the protocol. The full description of the protocol together with a replication package is available at the study website.[1]

4.1 Research Questions

We formulated the goal of the study using the classic **Goal-Question-Metric (GQM)** approach [92]:

Purpose: analyze and characterize
Issue: the use of machine learning
Object: to support self-adaptation based on MAPE-K feedback loops
Viewpoint: from a researcher's viewpoint.

We translated the overall goal of the review to three concrete research questions:

RQ1: What problems have been tackled by machine learning in self-adaptive systems?
RQ2: What are the key engineering aspects considered when applying learning in self-adaptation?
RQ3: What are open challenges for using machine learning in self-adaptive systems?

With RQ1, we wanted to gain insight into the motivations why machine learning has been applied in self-adaptation and, in particular, for what problems machine learning has been used in self-adaptive systems. With RQ2, we wanted to understand key aspects in the realization of self-adaptation. This included the MAPE functions of feedback loops that are supported by learning, the learning methods that have been used, and the representation of learning dimensions of these methods (as explained in the background section). With RQ3, we wanted to get insight into the limitations and the challenges in applying machine learning in self-adaptation that are reported in the studies.

[1] https://people.cs.kuleuven.be/danny.weyns/material/ML4SAS/SLR/.

4.2 Searched Sources and Search Query

The search strategy combined automatic with manual search. In a first step, we performed an automated search on the three main data search engines where research results on self-adaptation are published: IEEE Explore, ACM Digital Library, and Springer Link. To identify the search string and ensure that it finds all the relevant papers, we applied pilot searches for two main venues that publish research on self-adaptation, looking at the years 2017 till 2019: TAAS and SEAMS. During these pilots, we combined different keywords and compared the results of the query with manually identified papers. This way, we iteratively adjusted the search query such that the automatic search found all the manually identified papers with a minimum number of false positives. This resulted in the following search string that we used to select papers based on title and abstract:

(Title:learn AND (Title:adaptation OR Title:self*)) OR*
(Abstract:learn AND (Abstract:adaptation OR Abstract:self*))*

After finalizing the search string, we launched a full automated search to collect the papers (i.e., all the primary studies).[2] We then manually applied the inclusion and exclusion criteria to these papers to select relevant papers.

4.3 Inclusion and Exclusion Criteria

We used the following inclusion criteria to select papers:

- Papers that were published between January 2003 to May 2020[3]
- Papers that used machine learning methods in self-adaptive software systems that are based on a MAPE feedback loop. We target adaptation of application software or services that support the application software (in contrast to direct adaptation of hardware).
- Papers that provided a basic level of evaluation of the research, which may be in the form of a simple evaluation of application scenarios, a systematic simulation of a system, rigorous analysis, empirical evaluation, up to a real-world case study.

We used the following exclusion criteria:

- Surveys and roadmap papers, as we were only interested in studies that concretely apply machine learning in self-adaptive systems with a minimum level of evaluation.
- Tutorials, short papers,[4] editorials, and so on, because these papers do not provide sufficient data for the review.

A paper was selected if it met all inclusion criteria and did not meet any exclusion criterion.

4.4 Data Items

To answer the research questions, we defined a set of data items to be extracted from the papers. Table 2 gives an overview of the data items that we briefly discuss now. The concrete options for

[2]Note that we instantiated the search string according to the interface specific to each of the digital libraries we used. For instance, for the ACM library (https://dl.acm.org/search/advanced), the search query was formulated as "Title: (learn* AND (self* OR "adaptation")) OR Abstract: (learn* AND (self* OR "adaptation"))" and the publication date was set "From Jan. 2003 To May 2020." For a detailed description of the settings and process we used for each library, we refer to the study website.

[3]We selected 2003 as start date similar to previous reviews, based on the emergence of venues at the time that were dedicated to adaptation such as the International Conference on Autonomic Computing (ICAC).

[4]Papers with less than five pages are excluded.

Table 2. Data Extraction Items

Item ID	Item	Use
F1	Authors	Documentation
F2	Year	Documentation
F3	Title	Documentation
F4	Venue	Documentation
F5	Citation count	Documentation
F6	Quality score	RQ1-3
F7	Adaptation problem	RQ1
F8	Learning problem	RQ1
F9	MAPE function(s) supported by learning	RQ2
F10	Dimensions of learning methods	RQ2
F11	Learning method(s) used to support self-adaptation	RQ2
F12	Application domain	RQ1
F13	Limitations	RQ3
F14	Challenges	RQ3

each data item are further discussed in the next section. For a detailed description of the data items, we refer to the protocol that is available at the website of the systematic literature review.

F1–5: The data items author(s), year, title, venue, citation count are used for documentation purpose.

F6: Quality score assesses the quality of the reporting of the research in the paper, which is important for data analysis and interpretation of the results. Inline with References [22, 83], we assessed the following quality items: (1) problem definition of the study, (2) problem context, i.e., the way the study is related to other work, (3) research design, i.e., the way the study was organized, (4) contributions and study results, (5) insights derived from the study, (6) limitations of the study. For each item, we consider three quality levels: explicit description (2 points), general description (1 point), and no description (0 points). We calculate a quality assessment score (max 12) as the sum of the scores of all items of a study.

F7: The adaptation problem that is tackled by the managing system. Here, we look at what is the main problem for which self-adaptation is applied. The options are collected during data-gathering.

F8: The problem that is tackled by machine learning in the realization of self-adaptation. Here, we look at the motivations why machine learning is used to support self-adaptation, i.e., what concrete problem is solved. The options are collected during data-gathering.

F9: The MAPE functions supported by machine learning. Options are: monitor, analyzer, planner, executor, and any combination of the four basic functions.

F10: The dimensions of the learning methods applied (as explained in Section 2.2). Options for the dimensions are : unsupervised-interactive-supervised, active-passive, adversarial-non adversarial, and online-batch.

F11: The concrete machine learning methods used to solve learning problems, e.g., linear regression, support vector machine, reinforcement learning, classical neural network, deep neural network, and so on. The concrete options are collected during data-gathering.

F12: The application domain in which the machine learning method is applied to support self-adaptation. Initial options are: service-based system, cyber-physical system, internet-of-things, robotics, and cloud. Additional options are collected during data-gathering.

F13: The limitations reported in the paper. The options are collected during data-gathering.
F14: The challenges for future research on machine learning for self-adaptation reported in the paper. The options are collected during data-gathering.

4.5 Approach for Analysis

We tabulated the data in spreadsheets for analysis. We used descriptive statistics to present and analyze the quantitative aspects of the extracted data and summarize the data in a comprehensible format to answer the research questions. We presented results with plots using simple numbers and sometimes means and standard deviations to help understand the results. To analyze the data extracted for data items adaptation problem (F7), learning problem (F8), limitations (F13), and challenges (F14), we used open coding [30, 87, 94] to identify the categories for each data item. In particular, we collected short descriptions of characteristic fragments from the text in the papers and identified the general concepts by labeling occurrences. This allowed us to capture the essence of what problems have been tackled by machine learning to support self-adaptation and what open problems remain. Similar to others (e.g., Prechelt et al. [71]), we did not have a pre-defined coding schema, but we interpreted the text in the context of the specific data items. To avoid bias, we used the following process for coding. One researcher did a first preliminary coding. The results were then discussed among the three researchers and the codes were adjusted where needed based on consensus.

5 RESULTS

We now report the results. We start with some general information and then give results grouped per research question. All data and analysis results of the study are available at the study website.

5.1 Demographics

By applying the search string, we retrieved 6,709 papers. After applying the inclusion/exclusion criteria on these papers, we selected 109 papers for data collection.[5] These papers were published at 75 venues (conferences, journals, symposia, workshops, and books).

Figure 2 shows the number of selected papers distributed over the years of publication. The years before 2007 are not shown, since we did not find any papers before 2007. Twenty-eight percent of the studies were published between 2007 and 2014 and 72% between 2015 and 2019, which underpins the rapidly increasing research interest in the application of machine learning in self-adaptive systems.

The reporting quality of the research results for the selected papers is shown in Figure 3. The results show that most researchers provide a clear description of the problem they tackle and make clear how the problem relates to other work. Also, most of the papers clarify the contributions and discuss insights derived from the research, although not always explicitly. However, most studies ignore reporting limitations of the presented research. Similar results have been reported before in other systematic literature reviews; see, for instance, References [63, 105].

Table 3 gives an overview of the different venue types with the numbers of studies for each type and the venues with the highest number of papers in each category. The table also shows the mean and standard deviation of quality scores for the different types of venues. As we can observe,

[5]The main criteria for excluding papers were: no machine learning and no self-adaptive system (for instance, papers about self-learning in the context of e-learning and self-study), no self-adaptive system (for instance, papers presenting machine learning algorithms without any connection to self-adaptation), and no architecture-based adaptation (mostly papers about multi-agent systems that have no explicit distinction between managed system and managing system).

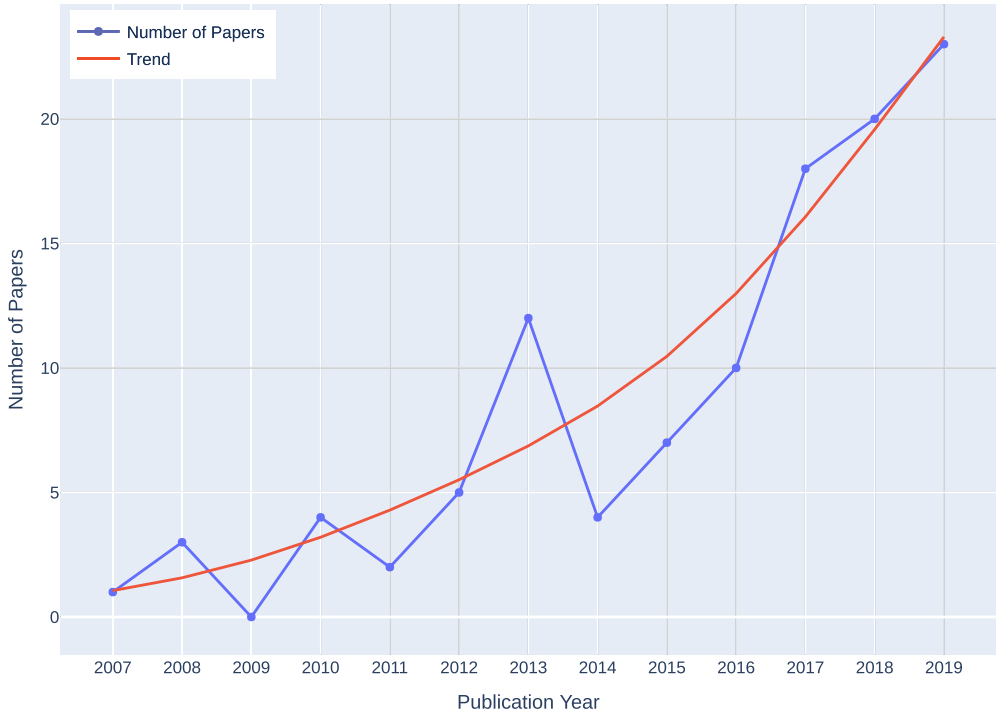

Fig. 2. Distribution of selected papers over the years.

Fig. 3. Quality scores of the selected papers.

the values confirm the common trend that the papers with the best quality scores are published in journals, while studies presented at workshops have lower quality scores. With a mean of the overall score of 7.7 (of 12) and a standard deviation of 1.9, the quality of the selected papers can be regarded as reasonably good, hence providing a basis for reliable data extraction.

Table 3. Venue Types with the Number of Studies and Top Venues per Type, and the Mean Values and Standard Deviations of the Quality Scores for the Different Venue Types

Venue Type	Number of Studies	Mean Quality Score	Standard Deviation Quality Score	Top Venues (with ≥ 3 studies)
Journals	36	8.6	1.7	TAAS (4) IEEE Access (4) TSE (3) Cluster Computing (3)
Conferences	50	7.0	1.7	ICAC (5) SASO (3)
Symposia	13	7.8	2.4	SEAMS (8)
Books	4	8.3	1.0	
Workshops	6	6.5	1.0	FAS*W (4)
Overall	109	7.7	1.9	

Data of venues with at least three papers among the selected papers; the number or papers for each top venue is specified inside parentheses.

5.2 RQ1: What problems have been tackled by machine learning in self-adaptive systems?

To answer this research question, we analyze the data of the following data items: Adaptation problem (F7), Learning problem (F8), Application domain (F12).[6]

Adaptation problem. In a concrete setting, self-adaptation is applied to solve a particular adaptation problem. This adaptation problem refers to the concerns that the managing system is dealing with. Previous research has shown that these concerns relate to quality properties of, and resources used by the system, see, e.g., References [93, 98]. Table 4 lists the adaptation problems we have identified from the papers, illustrated with examples. The last column shows the number of papers for each type.

Learning problem. A learning problem refers to a concrete problem that needs to be solved by machine learning in support to realize self-adaptation. Table 5 shows the six types of learning problems that we have identified from the papers. Each type is illustrated with examples. The last column shows the number of papers for each type of learning problem.

Adaptation problems versus Learning problems. We can now map adaptation problems to learning problems. This mapping allows us to identify whether particular types of adaptation problems delegate particular sub-problems to a machine learner. Figure 4 shows the mapping.

A few observations jump out. First, self-adaptive systems that aim at improving qualities of the managed system primarily use learning to solve the problem of updating and changing adaptation rules and policies. Second, self-adaptive systems that aim at protecting against cyber threats exploit learning only to detect or predict anomalies. Third, self-adaptive systems that aim at balancing qualities with resources used by the system exploit learners to solve all types of learning problems.

Application domain. Table 6 shows the application domains where machine learning has been applied and evaluated in support of self-adaptation. The results show that learning has been

[6]During the coding process, we used the following terminology. We used the term *quality* to refer to non-functional requirements that describe how a system should perform its functions, such as its performance and reliability. We used the term *resource* to refer to the means or supplies a software system uses to realize its functions, such as memory and bandwidth. We used the term *cost* to refer to monetary aspects, i.e., the price one has to pay to use or operate a system.

Table 4. Adaptation Problems Illustrated with Examples, with the Frequencies of Papers (Right Column)

Adaptation Problem	Brief Description with Example	#
Improve qualities	This adaptation problem is about improving (i.e., optimizing, maintaining, etc.) quality properties of the system. An example is keeping the response time low and the reliability high under changing workload and the occurrence of unexpected events [26].	44
Balance qualities with resources	This adaptation problem is about keeping a balance between improving quality properties of the system and resources (i.e., CPU, energy, etc.) required to achieve that improvement. For example, [62] aims at keeping the latency within a specified range while minimizing the computational resource required to achieve that.	37
Balance qualities with cost	This adaptation problem is about keeping a balance between improving the system's quality properties and the cost (i.e., operational, financial, etc.) required to achieve that. For example, [6] aims at keeping the failure rate of a service-based workflow below a specified threshold while minimizing the cost for using the services.	15
Improve resource allocation	This adaptation problem is about improving (i.e., optimizing, managing, etc.) the system's resource consumption. An example is managing the system's CPU and memory usage under uncertain workloads of the system [43].	10
Protect against cyber threats	This adaptation problem is about automating the cyber defense of a system by detecting and managing threats (i.e., intrusion, anomalies, etc.). For instance, [31] considers cyber defence in fifth-generation mobile systems by detecting and dealing with system intrusions.	3

applied in a wide variety of application domains, yet over 60% of the papers have studied and validated their work in three domains: cloud, client-server systems, and cyber-physical systems.

Application domains versus Adaptation problems. Figure 5 shows the adaptation problems solved in different application domains. Improving qualities is the main adaptation problem considered in all domains, except cloud. This can be expected as managing resources is vital in cloud applications, so balancing qualities with resources is the dominant adaptation problem in the cloud domain. Surprisingly, only a small fraction of the studies in the domains of cyber-physical systems and the internet-of-things consider resource management as part of the adaptation problem. Cost as an explicit factor in self-adaptation is mainly considered in client-server systems and service-based systems (in terms of the price to pay for using services).

Application domains versus Learning problems. It is also interesting to take a look at the mapping of solved learning problems in different application domains, as shown in Figure 6. One key observation is that learning for updating and changing adaptation rules and policies is used in all domains, with cloud and client-server systems as main domains. The latter resonates with the broad use of rule-based and policy-based techniques in the two domains. Learning to predict

Table 5. Learning Problems Illustrated with Examples, with the Frequencies of Papers (Right Column)

Learning Problem	Brief Description with Example	#
Update/Change adaptation rules/policies	This learning problem is about updating or changing adaptation rules or policies to support a managing system when dealing with changing operating conditions. For example, in [38], the managing system is supported by a reinforcement learner that dynamically updates adaptation policies to deal with changing workload.	36
Predict/Analyze resource usage	This learning problem is about predicting or analyzing resources that are used by the managed system that affect the decision-making of the managing system. Examples are learners that predict the energy consumption of batteries [59], storage [23], and CPU usage [23].	23
Keep runtime models up-to-date	This learning problem is about supporting a managing system with keeping runtime models up-to-date. Examples are a model of the environment [88], a performance model [46], and a reliability model [8].	18
Reduce large adaptation space	This learning problem is about supporting a managing system with reducing a large number of adaptation options (large adaptation space) such that the system can make more efficient decisions. For instance, [85] and [75] use learners to predict quality properties of adaptation options to select options, speeding up analysis.	16
Detect/Predict anomalies	This learning problem is about detecting or predicting anomalies in the behavior of the system or its environment that are relevant for adaptation. For example, in [55] a learner detects abnormal flow of traffic in a traffic management system and in [68] a learner identifies cyber threats in a communication network.	12
Collect unavailable prior knowledge	This learning problem is about collecting initially unknown runtime knowledge to support adaptation. For instance, in [37] a learner builds a performance model to support the managing system with computing the utility of different configurations; in [90] a learner identifies management policies without any prior knowledge.	4

and analyze resource usage, however, is primarily used in the cloud domain only. Learning to keep runtime models up to date as well as reducing large adaptation spaces are also broadly used across domains (except in network management). Finally, detecting and predicting anomalies is primarily applied in network management, but sporadically also in a variety of other domains.

Answer to RQ1—What problems have been tackled by machine learning in self-adaptive systems? We identified five types of adaptation problems where learning is applied: *improve qualities*, *balance qualities with resources*, *balance qualities with cost*, *improve resource allocation*, and *protect against cyber threats*. We identified six types of learning problems: *update/change adaptation rules/policies*, *predict/analyze resource usage*, *keep runtime models up-to-date*, *reduce large adaptation space*, *detect/predict anomalies*, and *collect unavailable prior knowledge*. The dominant case where learning is used to solve adaptation problems is updating and changing adaptation rules and policies to support improving qualities of the system. Learning to support self-adaptation has been applied in a variety of applications, with cloud, client-server system, and cyber-physical system as main domains.

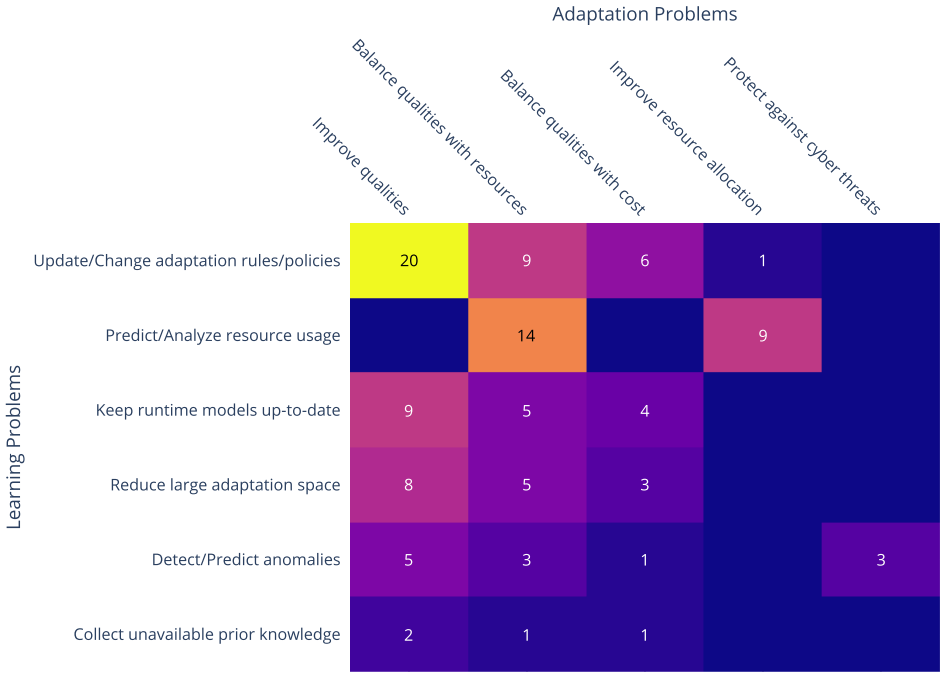

Fig. 4. Learning problems versus adaptation problems.

Table 6. Application Domains of the Collected Papers

Application domain	Number
Cloud	33
Client-server system	18
Cyber-physical system	16
Internet-of-things	9
Service-based system	8
Robotics	7
Network management	6
Business process management	4
Remote data mirroring	3
Traffic management	3
Stream processing	2
Grid computing	1
Medical simulation	1
No specific domain	3

5.3 RQ2: What are the key engineering aspects considered when applying learning in self-adaptation?

To answer this research question, we use the data items: MAPE stage(s) supported by learning (F9), Dimensions of learning methods (F10), and Learning methods used to support self-adaptation (F11).

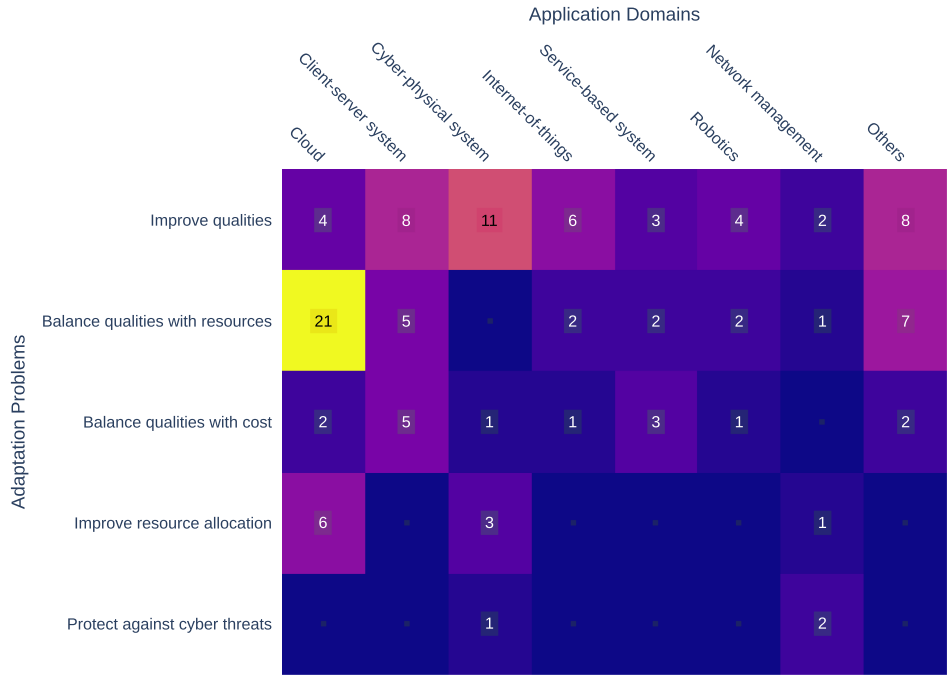

Fig. 5. Adaptation problems versus Application domains.

MAPE functions supported by learning. Figure 7 shows the distribution of learning methods used to support MAPE functions. The diagram shows that learning is dominantly used to support the analysis part of the decision-making process in the feedback loop (in 83 studies). Thirty-six of these studies apply learning in support of analysis only. A typical example is Reference [75], where machine learning is used to reduce large adaptation spaces such that only the relevant options need to be analyzed. Besides analysis, learning is often used to support planning of the feedback loop (57 studies). Fifteen of these studies apply learning in support of planning only. For example, Reference [67] adopted an instance-based learning method to implement a hybrid planning approach that selects an optimal planning strategy among possible strategies. Learning has also been used to support monitoring (23 studies), in particular, to update knowledge models. Ten of these studies apply learning in support to monitoring only. For example, in Reference [53], monitoring data is pre-processed using a light-weight classifier to learn optimization rules. We only identified one paper, Reference [68], where machine learning was used to support the execution function of the feedback loop (in combination with planning). In this paper, the authors used a classifier to choose the actuator that the system should use to adapt. The diagram shows that in a substantial number of papers (34 in total), learning supports both analysis and planning. An example is described in Reference [33], where machine learning is applied to generate Event-Condition-Action rules that are evaluated and subsequently used to make adaptation decisions. A smaller fraction of the papers (six in total) combine learning to support monitoring and analysis. Finally, a small number of papers (seven in total) apply learning that spans monitoring, analysis, and planning. As an example, Reference [3] proposed a proactive learner that supports monitoring, analysis, and planning by collecting context data, extracting required data for updating the learning model, and preparing a reasoning module for the decision-making process.

Learning problems versus MAPE functions. Figure 8 maps the learning problems to MAPE functions. The heat-map shows that all types of learning problems are tackled in support of

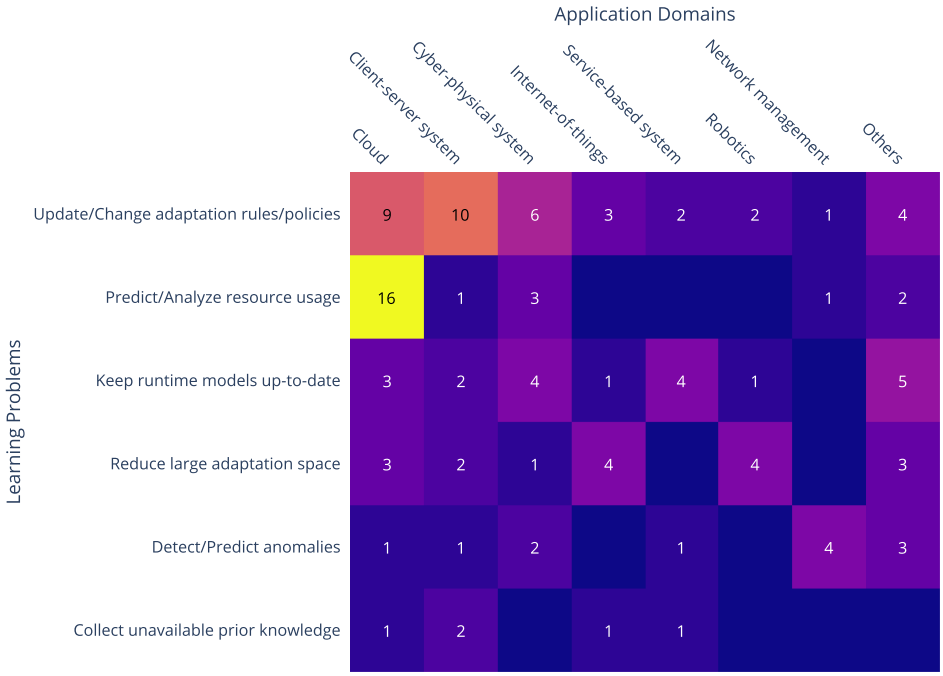

Fig. 6. Learning problems versus Application domains.

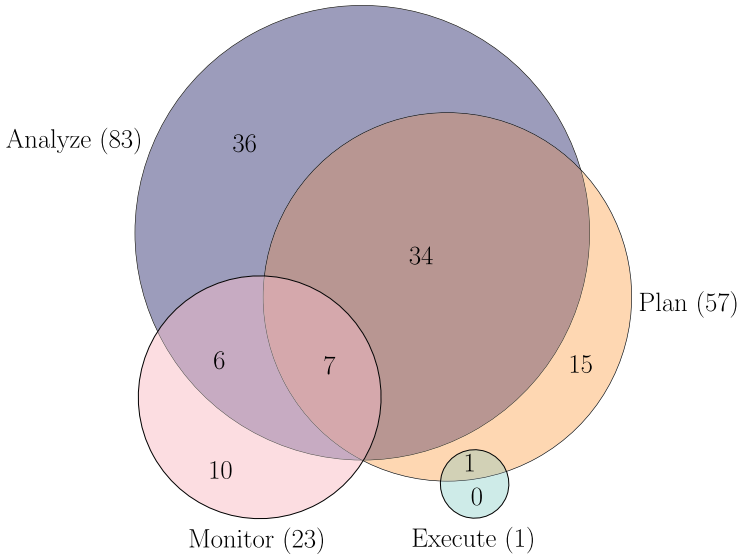

Fig. 7. Distribution of learning methods supporting the MAPE functions.

monitoring, analysis, and planning. As can be expected, the dominant case is learning used for updating and changing adaptation rules and policies to support analysis and planning, followed by predicting and analyzing resource usage. Another observation is the importance of learning used to keep runtime models up-to-date in support for monitoring and analysis.

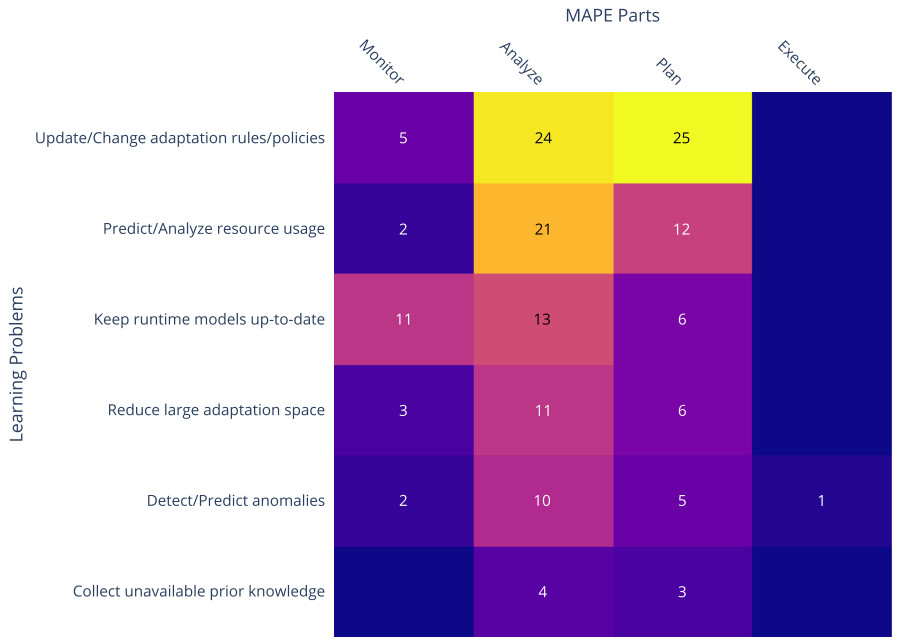

Fig. 8. Learning problems vs. MAPE functions.

Learning dimensions. Figure 9 gives an overview of the learning dimensions that have been applied in the papers. Each layer of the sunburst diagram shows the options for one of the learning dimensions, where the same value of a dimension is represented by the same color. The numbers in the diagram are based on the number of learning tasks that are solved to support self-adaptation. For instance, six learning tasks have been solved using one or more learning methods that apply online, non adversarial, passive, unsupervised learning. In total, 165 learning tasks have been solved in the 109 papers using a variety of learning methods. We zoom in on the concrete learning tasks solved by concrete learning methods below. The results show that a wide variety of combinations of dimensions are applied. The most popular learning methods used in self-adaptation apply supervised, passive, non adversarial, and online learning. In terms of learning type, we observe that supervised learning dominates (71% of the learning tasks). However, only a small number of papers apply unsupervised learning (7% of the learning tasks), which is surprising for self-adaptive systems that aim at automation and dealing with uncertainties that may not have been anticipated [7]. Example papers that applied unsupervised learning are [21] and [106], in particular, clustering-based learning techniques. Another observation is that we only encountered two papers that use adversarial learning, namely, References [51, 58] that applied a game-theoretical learning approach.

Learning problems versus learning types. Figure 10 shows the mapping of learning problems to learning types. The results demonstrate that supervised learning is frequently used for all types of learning problems, but mostly to predict and analyze resource usage. Interactive learning is also used for all types of learning problems, yet updating and changing adaptation rules and policies together with keeping runtime models up-to-date make up 80% of these problems. Unsupervised learning is most frequently used for detecting and predicting anomalies, but nevertheless supervised is still used three times more to tackle this learning problem.

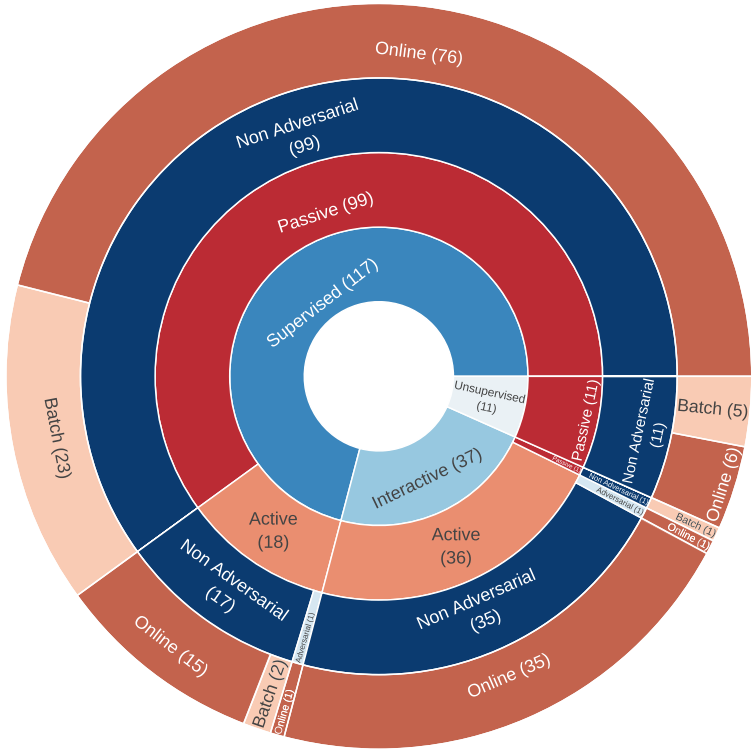

Fig. 9. Distribution of learning dimensions used in machine learning for self-adaptive systems.

Learning methods used to support self-adaptation. Figure 11 shows an overview of concrete learning methods used in self-adaptation. Note that multiple methods may be used in a single study. Each layer of the sunburst diagram shows a different level of abstraction of the learning methods. The inner layer groups methods based on learning type, i.e., the dimension "unsupervised vs. supervised vs. interactive" (we further elaborate on dimensions of the learning methods below). The layer in the middle groups learning methods based on common tasks of machine learning methods, i.e., classification, regression, reinforcement learning, clustering, and feature learning. This grouping is based on References [4, 80]. Note that one method can be used for multiple tasks, for instance support vector machines and deep learning have been used for both regression and classification. Finally, the outer layer shows concrete learning methods that were used in support of self-adaptation together with their frequencies (numbers between brackets). Areas marked with "Other tasks/methods" group other options. For instance, of the 37 papers with interactive learning methods, 31 use reinforcement learning to solve a learning problem; the other six studies use other methods, such as hidden semi-Markov models and partially observable Markov decision process. The diagram shows that the dominating learning method used in support of self-adaptation is model-free reinforcement learning (29 papers). An example is [2] that utilized fuzzy Q-learning to reason about new rules from the data collected at runtime. Other popular learning methods used in self-adaptation are support vector machines (15 times used; eight for regression and seven for classification), and traditional artificial neural networks and linear regression (both 14 times used). For instance, Reference [24] exploited a support vector machine to detect network attacks in cyber-physical systems, Reference [10] used an artificial neural network to predict qualities of services such as response time and throughput of the system, and Reference [18] applied linear regression to predict performance and power consumption.

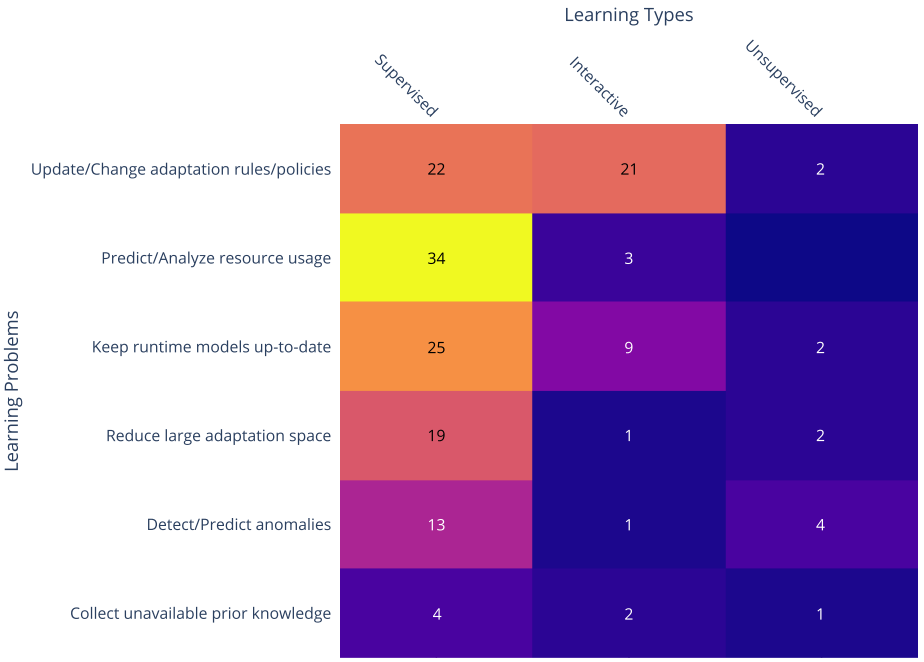

Fig. 10. Learning problems vs. Learning types.

Learning problems versus learning tasks. We also looked at the mapping between the learning problems in support of self-adaptation and the different types of learning tasks that need to be solved by the learners. Figure 12 shows an overview of this mapping. Regression is frequently used to solve all types of learning problems, but mostly to predict and analyze resource usages (36% of the problems solved with regression). Classification is also broadly used, with keeping runtime models up-to-date as the main learning problem (28% of the problems solved with a classifier). Updating and changing adaptation rules and policies is the primary learning problem solved by reinforcement learning (65% of the problems solved by a reinforcement learner).

Distribution of learning tasks over time and number of citations. To conclude, we look at the distribution of learning types over the years and the impact of the papers based on the number of citations they generated. For the latter, we used the citations of Google Scholar February 2021.[7] Figure 13 plots the results. We observe that only seven papers have generated more than 100 citations; four that used supervised learning [26, 28, 31, 108], two that used interactive learning [6, 90], and one paper that used unsupervised learning [31]. Overall, none of the three learning types seem to have generated clearly more impact (normalized number of citations[8] 2007–2019 for supervised learning average 4.8, standard 7.0; interactive learning average 4.4, standard 7.4; and unsupervised learning average 6.5, standard 11.4.). The plot shows that supervised and interactive learning have been used frequently over the full time span from 2007 till 2019. Yet, since 2016, we note an increase in the use of supervised learning and a decrease in interactive learning. Remarkably, after some small attention on the use of unsupervised learning in the period 2010 to 2013 (three studies), we observe an increase for this type of learning in the last three years, from 2017 to 2019 (eight studies).

[7]We used Google Scholar as it is widely used, but we acknowledge its limitations, such as the inclusion of self-citations.
[8]The number of citations for each paper has been normalized by past years since it was published, i.e., normalized number of citations of paper = Google Scholar citation of the paper in 2021/(2021—publication year of the paper).

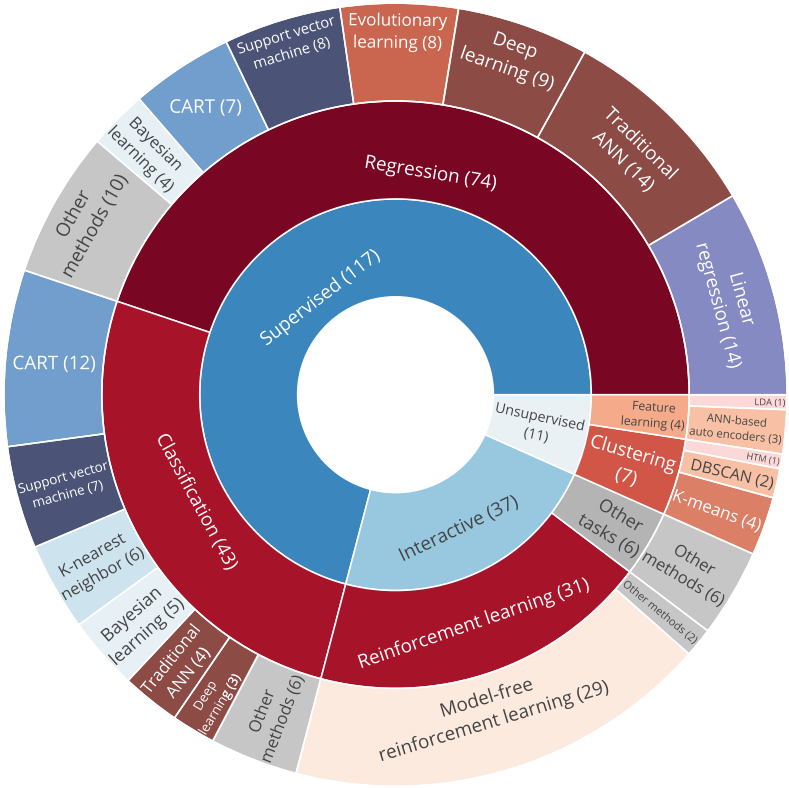

Fig. 11. Distribution of learning types, tasks, and methods used in self-adaptive systems. ANN refers to Artificial Neural Network, HTM to Hierarchical Temporal Memory, and LDA to Latent Dirichlet Allocation.

Answer to RQ2—What are the key engineering aspects considered when applying learning in self-adaptation? Machine learning is primarily used to support analysis and planning in self-adaptive systems. The majority of the papers use supervised or interactive learning; these learners typically exploit results of runtime analysis and observed effects of applied adaptations to learn. The most frequent problem of self-adaptation delegated to learning is updating and changing adaptation rules and policies (primarily solved using regression and reinforcement learning). Other important learning problems are predicting and analyzing resource usage (primarily solved using regression) and keeping runtime models up-to-date (primarily solved using regression and classification). The most popular learning method that is applied in self-adaptive systems is model-free reinforcement learning used for updating and changing adaptation rules and policies. Adversarial learning, however, is understudied, while this approach has a huge potential to deal with security concerns in self-adaptation. We observe that supervised and interactive learning have been used frequently over the years. Unsupervised learning, however, has only been used in a limited number of papers. Yet, this approach supports detecting novelty in data without any labeling, which can play a key role in managing complex types of uncertainty. None of the three types of learning has clearly generated more impact over the years.

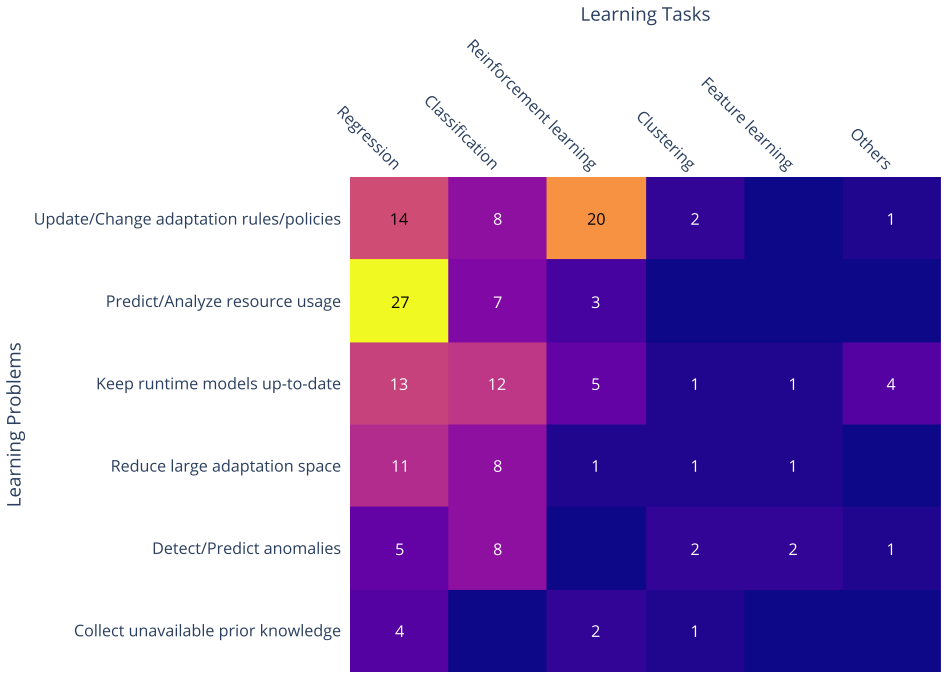

Fig. 12. Learning problems vs. Learning tasks.

5.4 RQ3: What are open challenges for using machine learning in self-adaptive systems?

To answer this research question, we analyze data items: Limitations (F13) and Challenges (F14).

Limitations. Table 7 lists the limitations reported in the papers. As illustrated in Figure 3, only a limited number of papers reported limitations of the applied learning methods. The results show that a variety of limitations of the learning methods have been reported. The most frequently reported limitation is limited scalability of the proposed learning approach. Other reported limitations relate to impact on qualities, in particular, performance and reusability, the scope in terms of uncertainty and guarantees that can be provided, and the need for expertise of humans to tune parameters.

Challenges. Table 8 lists the challenges reported in the papers. In total, 21 papers (19%) discussed challenges. Consequently, the reported challenges do not represent consensus nor importance of the challenges. However, most of the challenges apply to many other papers; yet, these authors have not explicitly mentioned them. The challenges are organized in five groups. Learning performance challenges primarily relate to timing aspects of learning. Learning effect challenges relate to uncertainties in terms of the effects of using learning in self-adaptation. Domain-related challenges are concerned with the characteristics of domains and the transfer of solutions to other problems. Policy-related challenges relate to the ability of learning methods to support the principles and rules for decision-making in self-adaptation. Finally, goal-related challenges relate to the need for machine learning techniques to support adaptation in practical systems that are characterized by multiple, possible evolving goals. We now zoom in on a few of the interesting open challenges and outline potential starting points to tackle them.

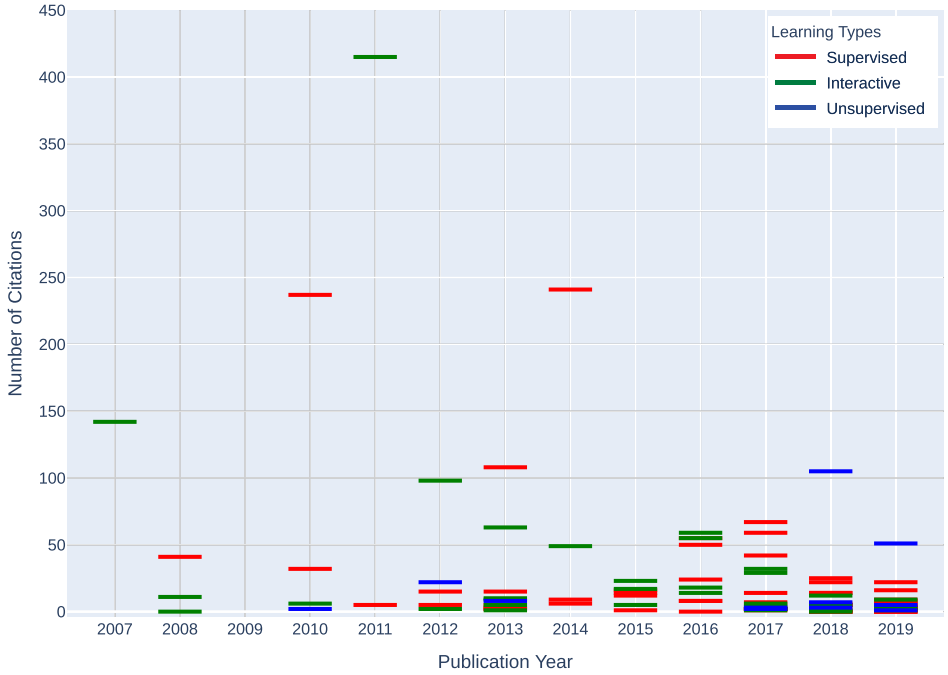

Fig. 13. Distribution of learning types through years and number of citations (Google Scholar 2/2022). Each study is represented by a tiny horizontal bar as indicated in the key. Studies of the same learning type with citation counts close to each other form thicker bars.

Table 7. Reported Limitations of the Learning Methods Applied in the Papers

Scalability	Learning approach is not scalable	6	[5, 8, 10, 68, 89, 90]
	No test of scalability due to data sensitivity	1	[77]
Performance	High computation time	3	[19, 56, 90]
	High computational load for feedback loop	1	[36]
	Slow convergence	1	[62]
Reusability	Solution is domain specific	2	[32, 36]
	Applicable for optimization problems only	1	[60]
Uncertainty	Cannot handle new situations	2	[29, 88]
	Cannot detect sudden changes	1	[89]
Guarantees	Optimization without satisfying all SLAs	1	[73]
	Might be trapped in a sub-optimal solution	1	[72]
Design	Need parameter tuning	4	[29, 31, 45, 62]

An open challenge in machine learning for self-adaptive systems is effect uncertainty [11, 72]. Effect uncertainty refers to uncertain effects on the system that may occur when a learner selects a configuration or a plan for adaptation that is applied on the system. Relying on the results of machine learning comes with some degree of (statistical) uncertainty that may affect the decision-making of a self-adaptive system. Detecting and handling this type of uncertainty is an open challenge. Note that this challenge is not specific to machine learning methods. However, we raise

Table 8. Open Challenges for Learning in Self-adaptation Reported in the Papers

Learning Performance	Balance time and accuracy	3	[66, 67, 84]
	Handle oscillations in early learning stages	1	[45]
Learning Effect	Understand the effect of learning on adaptation decisions over time	3	[11, 72, 107]
	Guarantees on results of machine learning	1	[75]
Domain-Related	Handle sudden changes	2	[70, 74]
	Handle open world changes	1	[95]
	Balancing diverse sources of input data	1	[86]
	Extend to other application domains	1	[18]
	Define similarity measures to transfer to other planning problems	1	[67]
Policy-Related	Deal with conflicting policies	2	[42, 70]
	Improve policy evolution speed	1	[40]
Goal-Related	Handle multiple goals	3	[28, 47, 75]
	Dynamically define utility function	1	[82]

it here as many studies have adopted machine learning methods for proactive decision making, where effect uncertainty gets more challenging by the uncertainty introduced by learning methods.

Learning about open-world changes is another open challenge in self-adaptive systems. Open world changes have been studied in the field of machine learning under the umbrella of "lifelong machine learning" [91], in particular, in relation to dealing with new learning tasks. A lifelong machine learner relies on an online learning pipeline that exploits historical knowledge to evaluate and update an existing learner to deal with new tasks. It may be possible to exploit such an approach to support a data-driven self-adaptive system with changes that were not fully anticipated.

In large-scale self-adaptive systems, the feedback loop's monitor component may be distributed over many different nodes (for instance sensor nodes) deployed on the managed system or in the environment. An example is described in Reference [104] where distributed monitoring components are used in online games that run on a client-server infrastructure. A crucial aspect of the sensed data on the decision-making for adaptation is the impact of heterogeneous sensor data [86], which may dynamically change. Automated weighting of heterogeneous data sources based on the current situation of the system to assure proper decision-making for adaptation is an open challenge.

One of the characteristic use cases of machine learning is reducing large adaptation spaces to support efficient analysis of different configurations based on model checking at runtime; see, for instance, Reference [75]. An open problem is to understand the impact of the learning process on the results of the model checker as this will affect the guarantees of the decisions made by the feedback loop. Such understanding will not only provide bounds on the expected impact of learning on the guarantees for decision-making in self-adaptive systems, it will also pave the way to dynamically balance the guarantees that are required with the resources that are available to provide them.

Transfer learning focuses on storing knowledge obtained from solving one problem and applying it to a different but related problem. For example, knowledge gained while learning to recognize anomalies in one type of communication network could then be exploited to recognize anomalies in another type of network. Transfer learning can help self-adaptation by reducing the cost of continuous training and data collection [47]. However, this study highlights that transfer learning has rarely been used in self-adaptation so far. Inspiration to tackle this open challenges is provided [40].

Another important open challenge when using leaning is handling multiple goals. Different approaches exist to deal with multiple goals in self-adaptation, such as utility functions [82], (semi-)ordered rules [75], and multi-objective functions [28]. A key issue of handling multiple goals is balancing time and recourse usage with finding a (close to) optimal solution. Hence, exploring the use of machine learning methods for efficient multi-objective optimization with guarantees on the precision of the results is an open challenge in machine learning for self-adaptation.

Answer to RQ3—What are open challenges for using machine learning in self-adaptive systems? Based on the reported limitations and challenges, we identified three broad categories of open challenges. The first category is about quality-related challenges. These include the scalability and performance of learning, and the reusability of solutions. The second category is about effect-related challenges. Central here are guarantees when learning is applied to support the feedback loop, uncertainties caused by learning and the effects of learning on decision-making. The third category is about design challenges. These include challenges related to the domain at hand and policy- and goal-related challenges.

6 INSIGHTS DERIVED FROM THE STUDY AND THREATS TO VALIDITY

Based on the insights derived from this systematic literature, we start this section by outlining an initial design process for applying machine learning in self-adaptive systems. Then, we discuss a number of remarkable observations of the survey that open interesting opportunities for future research. Finally, we discuss threats to validity of the research presented in this article.

6.1 Toward a Design Process for Using Machine Learning in Self-adaptive Systems

From the results of this review, we present an initial design process that can help guiding designers when applying machine learning in self-adaptive systems that are based on MAPE feedback loops. While resources exist that support engineers with the design of machine learning techniques in general, see for instance [20, 48], to the best of our knowledge, no such design process has been described for self-adaptive systems. Figure 14 shows the different elements of the design process we propose with the conceptual flow of activities between the elements.

The process starts with defining the adaptation problem that is posed by the domain (*1. Pose*). In this survey, we identified five types of adaptation problems that are solved by MAPE feedback loops supported by learning; see Table 4. These types can be instantiated for the problem at hand, supported by the data summarized in Figure 5. However, the list of adaptation problems can be extended when machine learning is applied to different types of domains and problems. In the next activity, the MAPE feedback loop of the managing system is designed that aims at solving the adaptation problem (*2. Design*). This means that the designer identifies the knowledge that is maintained by the feedback loop, the functionality that is required to monitor the managed system and its environment, to analyze the runtime data, to plan the actions for adaptation, and to enact these actions. Then, the designer uses domain knowledge (*3. Use*) to identify the learning problem that supports the MAPE feedback loop (*4. Identify*) (we assume for simplicity here that only a single learning problem needs to be solved by a single learner). The learning problem is delegated to, and solved by a machine learner. In this survey, we have identified six different types of learning problems in self-adaptive systems, as shown in Table 5. These types can be instantiated for the problem at hand. To that end, the designer can exploit the data summarized in Figures 4, 6, and 8. Similar to the list of adaptation problems, the list of learning problems can be extended

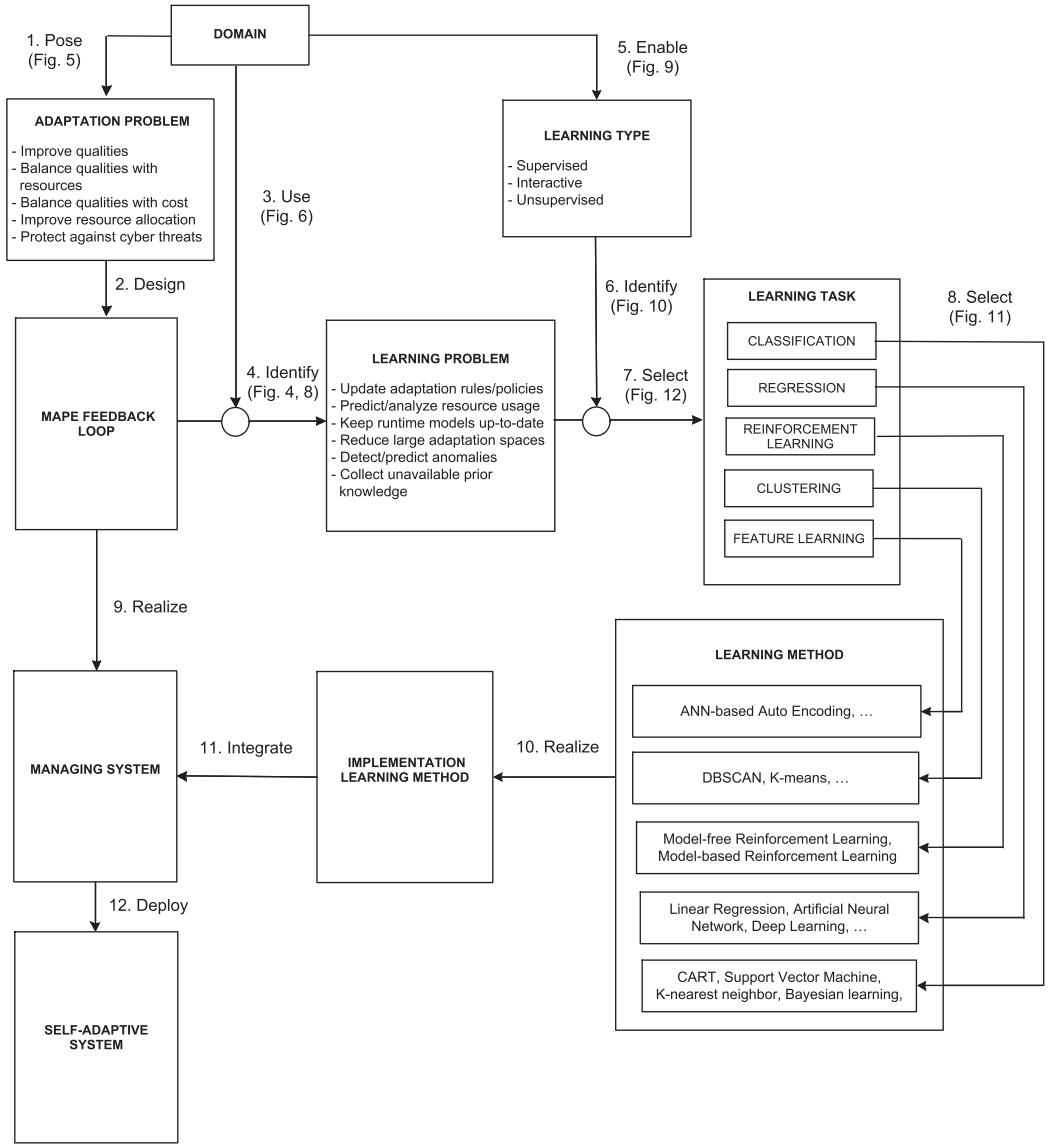

Fig. 14. Toward a design process for applying machine learning in self-adaptive systems.

when machine learning is applied to support MAPE feedback loops that deal with new types of adaptation problems in potentially new domains.

Next the learning task needs to be selected for the learning problem at hand. Besides the characteristics of the learning problem, this choice is determined by the learning type; for instance if no labeled data is available, supervised learning is not an option. The learning type depends both on the characteristics of the domain at hand (*5. Enable* and *6. Identify*) and the realization of the MAPE feedback loop. For the former, the designer can exploit the data summarized in Figures 9 and 10. For the mapping of the learning problem to the learning task (*7. Select*), the designer can exploit the data summarized in Figure 12. Next, a concrete learning method is selected to solve the

Table 9. Themes of Challenges with Concrete Focus as Reported in the Papers

Challenge Theme	Concrete Focus
Qualities	Scalability of learning, remove performance penalty
Uncertainty	Monitor uncertainty, detect novelty, support open world
Goals	Deal with changing goals, conflict of goals, new types of goals
Guarantees	Ensure quality goals, avoid sub-optimality, support explainability
Domain / Design	Deal with parameter tuning, transfer solutions, reusability of solution

Table 10. Additional Opportunities for Future Research Driven by Learning Methods

Learning Method	Concrete Opportunities
Unsupervised learning	Detecting new structures in complex data, support other learning methods
Active learning	Involve stakeholders in decision-making, reduce learning cost, increase speed of learning
Adversarial learning	Improve rules and policies, detect anomalies
Other learning methods	Detection of novel phenomena in environment, synchronize execution workflows in complex settings

learning task (*8. Select*). This literature review has identified a list of possible learning methods that have been applied to solve different types of learning tasks. The data summarized in Figure 11 supports the designer with selecting a learning method for the learning task at hand. Obviously, new methods can be added to this list as needed.

Finally, the managing system and the learning method are implemented (*9. Realize* and *10. Realize*, respectively) and the implementation of the learning method is integrated with the feedback loop to realize the managing system (*11. Integrate*). This system can then be tested and when accepted, it can be deployed to solve the adaptation problem of the self-adaptive system (*12. Deploy*).

The proposed process aims at providing a high-level outline of how machine learning can be integrated in the realization of MAPE-based self-adaptive systems. It is important to note that the flow of activities is conceptual; in practice different activities will be applied iteratively until the system is realized. Evidently, substantial effort will be required to turn this conceptual idea into a practical engineering process and support it with tools. We put this effort forward as a topic for future research in this area.

6.2 Opportunities for Future Research

The reported limitations of learning methods applied in self-adaptation (Table 7) as well as the open challenges for this area (Table 8) identify a number of shortcomings of existing learning approaches and highlight demands that may require the use of other or new learning methods to support self-adaptation. Table 9 summarizes the themes of the reported challenges.

The themes in Table 9 take the stance of the stakeholders of self-adaptive systems, looking from the perspective of the characteristics, demands, and open problems of these systems. We complement this view now with a set of additional opportunities for future research. To that end, we took a step back and explored prospects for advancing the field by looking at opportunities provided by learning methods. In particular, we looked at machine learning methods that received

less or no attention in existing work, and explored how self-adaptation may benefit from further investigation into the use of these learning methods. Table 10 summarizes the opportunities.

Only a small fraction (roughly 10%) of the papers apply unsupervised learning methods. This is remarkable given that one of the key drivers for applying self-adaptation is automating tasks in systems that are subject to uncertainty [96]. Since unsupervised learning methods can work independently of external input and do not require labeled data, we observe an interesting opportunity here to further explore unsupervised learning in self-adaptation. Unsupervised learning methods can be used as independent learning techniques to support self-adaptation; one interesting use case is the detection of new structures in complex high-dimensional data [25]. Unsupervised learning methods can also be used to support supervised or even interactive learning methods. A good example here is auto-encoders that have been used to increase the precision of other learning methods by providing a denser representation of data [31, 68, 103].

However, most papers apply passive learning. Active learning methods [79] interact with the environment or stakeholders to obtain the desired outputs at new data points. This helps to improve the performance of the machine learner by exploring the most informative data. In the context of self-adaptation, applying active learning provides an opportunity to involve stakeholders in the decision-making process, which has been highlighted as a key aspect of establishing trust [101]. Active learning can be exploited to reduce the learning cost and increase the convergence speed of learning. It can also be particularly useful to learn updating goals through interaction with stakeholders. In this way, the system can gradually learn essential knowledge of the stakeholder.

Adversarial learning aims at enabling a safe adoption of machine learning techniques in adversarial settings [57]. An adversarial machine learner tries to fool a learning model by supplying deceptive input. Adversarial learning can be particularly useful in domains that are sensitive to privacy and security issues, e.g., for signature detection and bio-metric recognition. Our study shows that only two papers have exploited non adversarial learning in self-adaptation that both adopt a game-theoretical learning approach. This observation opens opportunities to exploit various adversarial learning methods. One example is the use of generative adversarial networks to improve rules and policies in rule- and policy-based systems, or detect new anomalies for self-protection.

The majority of papers (85%) apply learning to support decision-making in self-adaptation, i.e., the analysis and planning functions. The main use cases are updating and changing rules and policies and predicting and analyzing resource usage. Only a limited number of papers (14%) apply learning to support monitoring, and here the main use case is keeping runtime models up-to-date. Only a single paper applies learning to support execution. This clearly opens opportunities for other use cases. One interesting opportunity is to use learning to support the detection of novel phenomena in the environment that have an effect on the self-adaptive system. Tackling this type of uncertainty is broadly seen as a key challenge in self-adaptive systems [7]. Another opportunity is to exploit learning in the execution of adaptation plans. In complex settings, for instance in large-scale applications with distributed feedback loops, the workflow and synchronisation of adaptation actions is often very difficult to establish manually. Machine learning can then be exploited to learn the best possible execution of the workflow under changing conditions.

6.3 Threats to Validity

We list the main threats to validity of this study and the measures we took to mitigate them.

Internal validity: refers to the extent to which a causal conclusion based on a study is warranted. Potential bias of reviewers is a common validity threat of literature reviews. For instance, a reviewer may be biased in the interpretation of fundamental concepts, i.e., machine learning and self-adaptive system. To mitigate this risk, we took two measures. First, the three researchers

involved in the study defined a protocol before starting the review process to clarify the definition of fundamental concepts and the process to follow. Second, the three researchers were involved in the selection of papers, the data collection and the analysis. A subset of the papers was handled independently by two researchers. The decisions on including or excluding papers and collecting data from the selected papers were based on an agreement between the two researchers. In case of disagreement, a third researcher was consulted, and after discussion, a decision was made in consensus.

External validity: refers to the generalizability of findings. Applied to this study, this threat is about generalization of the outcome and conclusions of the literature review. By limiting the automatic search to three online libraries, we may have missed some papers. To mitigate this threat, we applied the search string to the main libraries for publishing research in this area. This aligns with other literature reviews. In addition, we crosschecked that established venues for publishing papers in self-adaptation are covered. Furthermore, the search string we used may not provide the right coverage of papers. We mitigated this threat by starting the search process with pilot searches to define and tune the search string by collecting data from specific venues via the scientific search engines and comparing the results with manual inspection of the papers of the searched venues.

Construct validity: refers to the degree to which a study measures what it aims to measure. Here, the quality of reporting of studies may be a threat as this element affects the validity of the collected data. To anticipate this threat, we extracted data about reporting quality. The analysis of this data shows that the quality of reporting of the papers is of sufficient good quality. This result provides a solid basis to derive conclusions from extracted data. Moreover, to mitigate this threat, we excluded all short papers and papers that do not provide a minimum level of assessment. For instance, we excluded Reference [33], although the topic is relevant for our study, but this is a short paper. Similarly, we excluded Reference [81], since this regular paper does not provide a sufficient level of assessment.

Reliability: refers to assuring that the research findings can be replicated by another researcher. Here bias of researchers is also a potential validity threat. As explained above, to mitigate this threat, we defined a detailed protocol that provides the necessary guidelines for performing the different steps of the study. Multiple researchers did the paper selection, data extraction and analysis. Another technical threat concerns the methods used to collect papers from the search engines. For example, a search engine may change the operator to select any paper that complies with a query from a star (*) operator to an "ANY" operator (ignoring the initial version of the operator). To anticipate this threat, all the review material is available online, enabling a replication of the study.

7 CONCLUSION

This literature review aimed at shining a light on the state of the art of using machine learning in self-adaptive systems. The review confirms the rapidly growing research interests in this area. We identified six types of problems in self-adaptation that are solved by using machine learning: updating and changing adaptation rules and policies, predicting and analyzing resource usage, keeping runtime models up-to-date, reducing large adaptation spaces, detecting and predicting anomalies, and collecting unavailable prior knowledge. These problems are primarily solved to support analysis and planning in self-adaptation. Supervised and interactive learning dominate, primarily to solve regression, classification and reinforcement learning tasks. The reported limitations and challenges relate to quality properties when learning is used in self-adaptation, the effects of learning on the decision-making, and managing challenging aspects of the domain at hand.

From the data analysis, we identified an initial process to support designers that want to apply machine learning in the realization of self-adaptive system. We defined an open process that can be extended with new knowledge as we learn more about applying learning in self-adaptive systems.

Finally, we outlined a number of interesting opportunities for further research in this area, in particular, managing effect uncertainty, dealing with open world changes, dealing with distribution and heterogeneity of data, determining the bounds on guarantees for the adaptation goals implied by the use of machine learning, exploiting transfer learning to related problems, and finally dealing with more complex types of adaptation goals. We hope that the results of this systematic literature review will inspire researchers to tackle these and other problems in this fascinating research area.

APPENDIX

A LIST OF SELECTED PAPERS (PRIMARY STUDIES)

Title	Venue	Year
Comparison of Decision-Making Strategies for Self-Optimization in Autonomic Computing Systems	TAAS	2012
MARC: A Resource Consumption Modeling Service for Self-Aware Autonomous Agents	TAAS	2017
Fault Monitoring with Sequential Matrix Factorization	TAAS	2015
Generating Adaptation Rules of Software Systems: A Method Based on Genetic Algorithm	ICMLC	2018
To Adapt or Not to Adapt?: Technical Debt and Learning Driven Self-Adaptation for Managing Runtime Performance	ICPE	2018
Adaptive model learning for continual verification of non-functional properties	ICPE	2014
SATISFy: Towards a Self-Learning Analyzer for Time Series Forecasting in Self-Improving Systems	FAS*W	2018
A Concept for Proactive Knowledge Construction in Self-Learning Autonomous Systems	FAS*W	2018
Meta-Learning for Realizing Self-x Management of Future Networks	IEEE Access	2017
Framework for Building Self-Adaptive Component Applications Based on Reinforcement Learning	SCC	2018
Introducing Deep Learning Self-Adaptive Misuse Network Intrusion Detection Systems	IEEE Access	2019
A Self-Adaptive Deep Learning-Based System for Anomaly Detection in 5G Networks	IEEE Access	2018
Self-Adaptation Applied to MQTT via a Generic Autonomic Management Framework	ICIT	2019
Instance-Based Learning for Hybrid Planning	FAS*W	2017
An algorithm for online planning to improve availability and performance of self-adaptive websites	CFIS	2017
Towards History-Aware Self-Adaptation with Explanation Capabilities	FAS*W	2019
Protecting Cyber Physical Systems Using a Learned MAPE-K Model	IEEE Access	2019
Reinforcement Learning-Based Predictive Control for Autonomous Electrified Vehicles	IV	2018
Machine Learning Meets Quantitative Planning: Enabling Self-Adaptation in Autonomous Robots	SEAMS	2019
A Reinforcement Learning-Based Framework for the Generation and Evolution of Adaptation Rules	ICAC	2017
A Learning Approach to Enhance Assurances for Real-Time Self-Adaptive Systems	SEAMS	2018
Adding Self-Improvement to an Autonomic Traffic Management System	ICAC	2017
A self-adaptation framework for dealing with the complexities of software changes	ICSESS	2017
Losing Control: The Case for Emergent Software Systems Using Autonomous Assembly, Perception, and Learning	SASO	2016
Two-Level Autonomous Optimizations Based on ML for Cardiac FEM Simulations	ICAC	2018
Adaptive runtime response time control in PLC-based real-time systems using reinforcement learning	SEAMS	2018
Adaptivity at every layer: a modular approach for evolving societies of learning autonomous systems	SEAMS	2008
Autonomic Software Product Lines (ASPL)	ECSA	2010
Learning to sample: exploiting similarities across environments to learn performance models for configurable systems	FSE	2018
Learning revised models for planning in adaptive systems	ICSE	2013
Knowledge Base K Models to Support Trade-Offs for Self-Adaptation using Markov Processes	SASO	2019
Learning a Dynamic Re-combination Strategy of Forecast Techniques at Runtime	ICAC	2015
Efficient analysis of large adaptation spaces in self-adaptive systems using machine learning	SEAMS	2019
A Learning-Based Framework for Engineering Feature-Oriented Self-Adaptive Software Systems	TSE	2013
Transfer Learning for Improving Model Predictions in Highly Configurable Software	SEAMS	2017
An Analysis of Decision-Making Techniques in Dynamic, Self-Adaptive Systems	SASO	2014
All Versus One: An Empirical Comparison on Retrained and Incremental Machine Learning for Modeling Performance of Adaptable Software	SEAMS	2019
Training Prediction Models for Rule-Based Self-Adaptive Systems	ICAC	2018
Learning non-deterministic impact models for adaptation	SEAMS	2018
FUSION: a framework for engineering self-tuning self-adaptive software systems	FSE	2010
RaM: Causally-Connected and Requirements-Aware Runtime Models using Bayesian Learning	MODELS	2019
Adaptive Execution of Continuous and Data-intensive Workflows with Machine Learning	Middleware	2018
Fuzzy Self-Learning Controllers for Elasticity Management in Dynamic Cloud Architectures	QoSA	2016
Handling Uncertainty in Self-Adaptive Software Using Self-Learning Fuzzy Neural Network	COMPSAC	2016
Supporting the Self-Learning of Systems at the Network Edge with Microservices	SSI	2019
Multi-Model Deep Learning for Cloud Resources Prediction to Support Proactive Workflow Adaptation	IEEE Cloud Summit	2019
A self-learning strategy for artificial cognitive control systems	INDIN	2015
SLOPE: A Self Learning Optimization and Prediction Ensembler for Task Scheduling	WiMob	2018
Self-Learning Production Systems (SLPS)—Energy management application for machine tools	ISIE	2013
Machine Learning for Achieving Self-* Properties and Seamless Execution of Applications in the Cloud	NCCA	2015
Driving skill analysis using machine learning The full curve and curve segmented cases	ITST	2012
Self-Learning approach to support lifecycle optimization of Manufacturing processes	IECON	2013
Managing Uncertainty in Autonomic Cloud Elasticity Controllers	IEEE Cloud Computing	2016
Self-Adaptive and Online QoS Modeling for Cloud-Based Software Services	TSE	2017
Elasticat: A load rebalancing framework for cloud-based key-value stores	HiPC	2012
Energy Efficiency in Machine Tools - A Self-Learning Approach	SMC	2013
Model-based reinforcement learning approach for planning in self-adaptive software system	IMCOM	2015

Self-evolvable knowledge-enhanced IoT data mobility for smart environment	IML	2017
TSLAM: A Trust-enabled Self-Learning Agent Model for Service Matching in the Cloud Market	TAAS	2019
SLICE: self-learnable IoT common software engine	IOT	2018
An adaptive prediction approach based on workload pattern discrimination in the cloud	JNCA	2017
A Reconfiguration Algorithm for Power-Aware Parallel Applications	TACO	2016
Effective Decision Making in Self-adaptive Systems Using Cost-Benefit Analysis at Runtime and Online Learning of Adaptation Spaces	ENASE	2019
Self-Learning Production Systems: Adapter Reference Architecture	FAIM	2013
Decentralized Planning for Self-Adaptation in Multi-cloud Environment	ESOCC	2015
IDES: Self-adaptive Software with Online Policy Evolution Extended from Rainbow	Computer and Information Science (Book)	2012
Rationalism with a dose of empiricism: combining goal reasoning and case-based reasoning for self-adaptive software systems	Requirements Engineering Journal	2015
Adaptive process control based on a self-learning mechanism in autonomous manufacturing systems	JAMT	2012
A Q-Leaning-Based On-Line Planning Approach to Autonomous Architecture Discovery for Self-managed Software	OTM	2008
A three-phase decision making approach for self-adaptive systems using web services	CASM	2018
Architectural Homeostasis in Self-Adaptive Software-Intensive Cyber-Physical Systems	ECSA	2016
Towards Self-adaptation Planning for Complex Service-Based Systems	ICSOC	2013
Risk management for self-adapting self-organizing emergent multi-agent systems performing dynamic task fulfillment	AAMAS	2014
Synthesis and Verification of Self-aware Computing Systems	Self-Aware Computing Systems (Book)	2017
PRESC 22 : efficient self-reconfiguration of cache strategies for elastic caching platforms	Computing	2013
Avionics Self-adaptive Software: Towards Formal Verification and Validation	ICDCIT	2019
Self-* programming: run-time parallel control search for reflection box	Evolving Systems	2013
A Model-Based Approach to Dynamic Self-assessment for Automated Performance and Safety Awareness of Cyber-Physical Systems	IMBSA	2017
Automatic Adaptation of SOA Systems Supported by Machine Learning	DoCEIS	2013
VM Reservation Plan Adaptation Using Machine Learning in Cloud Computing	Journal of Grid Computing	2019
Machine learning-based auto-scaling for containerized applications	Neural Computing and Applications (Journal)	2019
A Model for Using Machine Learning in Smart Environments	GPC	2011
A cognitive/intelligent resource provisioning for cloud computing services: opportunities and challenges	Soft Computing	2019
An autonomic resource provisioning framework for efficient data collection in cloudlet-enabled wireless body area networks: a fuzzy-based proactive approach	Soft Computing	2019
An autonomic approach for resource provisioning of cloud services	Cluster Computing	2016
Toward Proactive Learning of Multi-layerd Cloud Service Based Application	CLOSER	2016
Utilizing Twitter Data for Identifying and Resolving Runtime Business Process Disruptions	OTM	2018
A Self Healing Microservices Architecture: A Case Study in Docker Swarm Cluster	AINA	2019
Performance Comparison of Deep VM Workload Prediction Approaches for Cloud	ICCAN	2018
Towards Automated Analysis and Optimization of Multimedia Streaming Services Using Clustering and Semantic Techniques	MACE	2010
Building Autonomic Elements from Video-Streaming Servers	JNSM	2019
PSO-based novel resource scheduling technique to improve QoS parameters in cloud computing	Neural Computing and Applications (Journal)	2019
On the use of hybrid reinforcement learning for autonomic resource allocation	Cluster Computing	2007
IO dependent SSD cache allocation for elastic Hadoop applications	SCIS (Jorunal)	2018
Architecting Dependable Systems with Proactive Fault Management	Architecting Dependable Systems VII (Book)	2010
An autonomic provisioning framework for outsourcing data center based on virtual appliances	Cluster Computing	2008
An Auto-Scaling Cloud Controller Using Fuzzy Q-Learning - Implementation in OpenStack	ESOCC	2016
Mobile Apps with Dynamic Bindings Between the Fog and the Cloud	ICSOC	2019
A Self-Learning Scheduling in Cloud Software Defined Block Storage	CLOUD	2017
Self-Adaptive Learning PSO-Based Deadline Constrained Task Scheduling for Hybrid IaaS Cloud	TASE	2014
Dynamic adaptation of policies using machine learning	CCGRID	2016
A self-learning approach for validation of runtime adaptation in service-oriented systems	SOCA	2017
Self-adaptation for Mobile Robot Algorithms Using Organic Computing Principles	ARCS	2013
Using a recurrent artificial neural network for dynamic self-adaptation of cluster-based web-server systems	APIN	2017
A Control-Theoretic Approach to Self-adaptive Systems and an Application to Cloud-Based Software	LASER	2013
A Nash equilibrium based decision-making method for internet of things	JAIHC	2019
Building Automated Data Driven Systems for IT Service Management	JNSM	2017
Catch-up TV forecasting: enabling next-generation over-the-top multimedia TV services	MTAP	2017
Dynamic QoS Management and Optimization in Service-Based Systems	TSE	2011

REFERENCES

[1] J. Hellerstein, Y. Diao, S. Parekh, and D. Tilbury. 2004. *Feedback Control of Computing Systems*. John Wiley and Sons, New York. https://doi.org/10.1002/047166880X

[2] H. Arabnejad, P. Jamshidi, G. Estrada, N. El Ioini, and C. Pahl. 2016. An auto-scaling cloud controller using fuzzy q-learning-implementation in openstack. In *Proceedings of the European Conference on Service-Oriented and Cloud Computing*. Springer, 152–167.

[3] K. Bierzynski, P. Lutskov, and U. Assmann. 2019. Supporting the self-learning of systems at the network edge with microservices. In *Proceedings of the 13th International Conference and Exhibition on Integration Issues of Miniaturized Systems: Smart Systems Integration*. 1–8.
[4] C. Bishop. 2006. *Pattern Recognition and Machine Learning*. Springer.
[5] R. Calinescu, M. Autili, J. Cámara, A. Di Marco, S. Gerasimou, P. Inverardi, A. Perucci, N. Jansen, J.-P. Katoen, M. Kwiatkowska, O. Mengshoel, R. Spalazzese, and M. Tivoli. 2017. *Synthesis and Verification of Self-aware Computing Systems*. Springer International Publishing, Cham, 337–373. https://doi.org/10.1007/978-3-319-47474-8_11
[6] R. Calinescu, L. Grunske, M. Kwiatkowska, R. Mirandola, and G. Tamburrelli. 2011. Dynamic QoS management and optimization in service-based systems. *IEEE Trans. Softw. Eng.* 37, 3 (2011), 387–409. https://doi.org/10.1109/TSE.2010.92
[7] R. Calinescu, R. Mirandola, D. Perez-Palacin, and D. Weyns. 2020. Understanding uncertainty in self-adaptive systems. In *Proceedings of the IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS'20)*. 242–251. https://doi.org/10.1109/ACSOS49614.2020.00047
[8] R. Calinescu, Y. Rafiq, K. Johnson, and M. Bakundefinedr. 2014. Adaptive model learning for continual verification of non-functional properties. In *Proceedings of the 5th ACM/SPEC International Conference on Performance Engineering (ICPE'14)*. Association for Computing Machinery, New York, NY, 87–98. https://doi.org/10.1145/2568088.2568094
[9] J. Cámara, D. Garlan, G. Moreno, and B. Schmerl. 2017. Analyzing self-adaptation via model checking of stochastic games. In *Software Engineering for Self-Adaptive Systems III. Assurances*. Springer.
[10] T. Chen and R. Bahsoon. 2017. Self-adaptive and online qos modeling for cloud-based software services. *IEEE Trans. Softw. Eng.* 43, 5 (2017), 453–475.
[11] T. Chen, R. Bahsoon, S. Wang, and X. Yao. 2018. To adapt or not to adapt? technical debt and learning driven self-adaptation for managing runtime performance. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering* (Berlin, Germany) *(ICPE'18)*. Association for Computing Machinery, New York, NY, 48–55. https://doi.org/10.1145/3184407.3184413
[12] B. Cheng, R. de Lemos, H. Giese, P. Inverardi, J. Magee, J. Andersson, B. Becker, N. Bencomo, Y. Brun, B. Cukic, G. Di Marzo Serugendo, S. Dustdar, A. Finkelstein, C. Gacek, K. Geihs, V. Grassi, G. Karsai, H. Kienle, J. Kramer, M. Litoiu, S. Malek, R. Mirandola, H. Müller, S. Park, M. Shaw, M. Tichy, M. Tivoli, D. Weyns, and J. Whittle. 2009. Software engineering for self-adaptive systems: A research roadmap. In *Software Engineering for Self-Adaptive Systems*. Springer, 1–26. https://doi.org/10.1007/978-3-642-02161-9_1
[13] B. H. C. Cheng, A. Ramirez, and P. K. McKinley. 2013. Harnessing evolutionary computation to enable dynamically adaptive systems to manage uncertainty. In *Proceedings of the International Workshop on Combining Modelling and Search-Based Software Engineering (CMSBSE'13)*. 1–6. https://doi.org/10.1109/CMSBSE.2013.6604427
[14] L. Cui, S. Yang, F. Chen, Z. Ming, N. Lu, and J. Qin. 2018. A survey on application of machine learning for Internet of Things. *Int. J. Mach. Learn. Cybernet.* 9, 8 (2018), 1399–1417.
[15] J. Cámara, P. Correia, R. de Lemos, D. Garlan, P. Gomes, B. Schmerl, and R. Ventura. 2016. Incorporating architecture-based self-adaptation into an adaptive industrial software system. *J. Syst. Softw.* 122 (2016), 507–523. https://doi.org/10.1016/j.jss.2015.09.021
[16] C. da Silva, J. da Silva, C. Paterson, and R. Calinescu. 2017. Self-adaptive role-based access control for business processes. In *Proceedings of the 12th IEEE/ACM International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. https://doi.org/10.1109/SEAMS.2017.13
[17] M. D'Angelo, S. Gerasimou, S. Ghahremani, J. Grohmann, I. Nunes, E. Pournaras, and S. Tomforde. 2019. On learning in collective self-adaptive systems: State of practice and a 3D framework. In *Proceedings of the IEEE/ACM 14th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS'19)*. 13–24. https://doi.org/10.1109/SEAMS.2019.00012
[18] D. De Sensi, M. Torquati, and M. Danelutto. 2016. A reconfiguration algorithm for power-aware parallel applications. *ACM Trans. Architect. Code Optimiz.* 13, 4 (2016), 1–25.
[19] A. Dhrgam, D. Kim, and L. Lu. 2018. A three-phase decision making approach for self-adaptive systems using web services. *Complex Adapt. Syst. Model.* 6, 1 (2018), 8.
[20] P. Domingos. 2012. A few useful things to know about machine learning. *Commun. ACM* 55, 10 (2012), 78–87. https://doi.org/10.1145/2347736.2347755
[21] F. Duarte, R. Gil, P. Romano, A. Lopes, and L. Rodrigues. 2018. Learning non-deterministic impact models for adaptation. In *Proceedings of the 13th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. https://doi.org/10.1145/3194133.3194138
[22] T. Dybå and T. Dingsøyr. 2008. Empirical studies of agile software development: A systematic review. *Info. Softw. Technol.* 50, 9–10 (2008), 833–859.
[23] H. El-Kassabi, M. A. Serhani, S. Bouktif, and A. Benharref. 2019. Multi-model deep learning for cloud resources prediction to support proactive workflow adaptation. In *Proceedings of the IEEE Cloud Summit*. 78–85.

[24] I. Elgendi, M. F. Hossain, A. Jamalipour, and K. S. Munasinghe. 2019. Protecting cyber physical systems using a learned MAPE-K model. *IEEE Access* 7 (2019), 90954–90963. https://doi.org/10.1109/ACCESS.2019.2927037

[25] E. Elhamifar and R. Vidal. 2013. Sparse subspace clustering: Algorithm, theory, and applications. *IEEE Trans. Pattern Anal. Mach. Intell.* 35, 11 (2013), 2765–2781. https://doi.org/10.1109/TPAMI.2013.57

[26] A. Elkhodary, N. Esfahani, and S. Malek. 2010. FUSION: A framework for engineering self-tuning self-adaptive software systems. In *Proceedings of the 8th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. https://doi.org/10.1145/1882291.1882296

[27] I. Epifani, C. Ghezzi, R. Mirandola, and G. Tamburrelli. 2009. Model evolution by run-time parameter adaptation. In *Proceedings of the 31st International Conference on Software Engineering (ICSE'09)*. IEEE Computer Society, Washington, DC, 111–121. https://doi.org/10.1109/ICSE.2009.5070513

[28] N. Esfahani, A. Elkhodary, and S. Malek. 2013. A learning-based framework for engineering feature-oriented self-adaptive software systems. *IEEE Trans. Softw. Eng.* 39, 11 (Nov. 2013), 1467–1493. https://doi.org/10.1109/TSE.2013.37

[29] D. Feng and C. Germain. 2015. Fault monitoring with sequential matrix factorization. *ACM Trans. Auton. Adapt. Syst.* 10, 3, Article 20 (Oct. 2015), 25 pages. https://doi.org/10.1145/2797141

[30] D. Mendez Fernandez, S. Wagner, M. Kalinowski, M. Felderer, P. Mafra, A. Vetro, T. Conte, M. Christiansson, D. Greer, C. Lassenius, T. Mannisto, M. Nayabi, M. Oivo, B. Penzenstadler, and D. Pfahl. 2016. Naming the pain in requirements engineering. *Empir. Softw. Eng.* 22 (2016), 2298–2338.

[31] L. Fernandez Maimo, Angel L. Perales Gomez, F. J. Garcia Clemente, M. Gil Perez, and G. Martinez Perez. 2018. A self-adaptive deep learning-based system for anomaly detection in 5G networks. *IEEE Access* 6 (2018), 7700–7712. https://doi.org/10.1109/ACCESS.2018.2803446

[32] M. Ferroni, A. Corna, A. Damiani, R. Brondolin, J. Kubiatowicz, D. Sciuto, and M. Santambrogio. 2017. MARC: A resource consumption modeling service for self-aware autonomous agents. *ACM Trans. Auton. Adapt. Syst.* 12, 4 (2017). https://doi.org/10.1145/3127499

[33] A. Frömmgen, R. Rehner, M. Lehn, and A. Buchmann. 2015. Fossa: Learning ECA rules for adaptive distributed systems. In *Proceedings of the IEEE International Conference on Autonomic Computing*. 207–210. https://doi.org/10.1109/ICAC.2015.37

[34] Alessio Gambi, Giovanni Toffetti, and Mauro Pezzè. 2013. *Assurance of Self-adaptive Controllers for the Cloud*. Springer Berlin Heidelberg, Berlin, Heidelberg, 311–339. https://doi.org/10.1007/978-3-642-36249-1_12

[35] D. Garlan, S.-W. Cheng, A.-C. Huang, B. Schmerl, and P. Steenkiste. 2004. Rainbow: Architecture-based self-adaptation with reusable infrastructure. *Computer* 37, 10 (2004), 46–54.

[36] I. Gerostathopoulos, D. Skoda, F. Plasil, T. Bures, and A. Knauss. 2016. Architectural homeostasis in self-adaptive software-intensive cyber-physical systems. In *Software Architecture*. Springer.

[37] S. Ghahremani, C. M. Adriano, and H. Giese. 2018. Training prediction models for rule-based self-adaptive systems. In *Proceedings of the IEEE International Conference on Autonomic Computing (ICAC'18)*. 187–192. https://doi.org/10.1109/ICAC.2018.00031

[38] X. Gu. 2012. *IDES: Self-adaptive Software with Online Policy Evolution Extended from Rainbow*. Springer, Berlin, 181–195. https://doi.org/10.1007/978-3-642-30454-5_13

[39] F. Heylighen. 2002. The science of self-organization and adaptivity. In *Knowledge Management, Organizational Intelligence and Learning, and Complexity: vol. 3*, L. D. Kiel (Ed.). EOLSS Publishers.

[40] H. Ho and E. Lee. 2015. Model-based reinforcement learning approach for planning in self-adaptive software system. In *Proceedings of the 9th International Conference on Ubiquitous Information Management and Communication*. https://doi.org/10.1145/2701126.2701191

[41] U. Iftikhar and D. Weyns. 2014. ActivFORMS: Active formal models for self-adaptation. In *Proceedings of the 9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS'14)*. https://doi.org/10.1145/2593929.2593944

[42] A. Ismail and V. Cardellini. 2015. Decentralized planning for self-adaptation in multi-cloud environment. In *Advances in Service-Oriented and Cloud Computing*, G. Ortiz and C. Tran (Eds.). Springer.

[43] P. Jamshidi, C. Pahl, and N. C. Mendonca. 2016. Managing uncertainty in autonomic cloud elasticity controllers. *IEEE Cloud Comput.* 3, 3 (2016), 50–60. https://doi.org/10.1109/MCC.2016.66

[44] P. Jamshidi, A. Sharifloo, C. Pahl, H. Arabnejad, A. Metzger, and G. Estrada. 2016. Fuzzy self-learning controllers for elasticity management in dynamic cloud architectures. In *Proceedings of the 12th International ACM SIGSOFT Conference on Quality of Software Architectures*. 70–79. https://doi.org/10.1109/QoSA.2016.13

[45] P. Jamshidi, A. Sharifloo, C. Pahl, H. Arabnejad, A. Metzger, and G. Estrada. 2016. Fuzzy self-learning controllers for elasticity management in dynamic cloud architectures. In *Proceedings of the 12th International ACM SIGSOFT Conference on Quality of Software Architectures (QoSA'16)*. 70–79. https://doi.org/10.1109/QoSA.2016.13

[46] P. Jamshidi, M. Velez, C. Kästner, and N. Siegmund. 2018. Learning to sample: Exploiting similarities across environments to learn performance models for configurable systems. In *Proceedings of the 26th ACM Joint Meeting*

on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. https://doi.org/10.1145/3236024.3236074

[47] P. Jamshidi, M. Velez, C. Kastner, N. Siegmund, and P. Kawthekar. 2017. Transfer learning for improving model predictions in highly configurable software. In *Proceedings of the IEEE/ACM 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS'17)*. 31–41. https://doi.org/10.1109/SEAMS.2017.11

[48] D. Julian. 2016. *Designing Machine Learning Systems with Python*. PACKT. Retrieved from https://www.packtpub.com/product/designing-machine-learning-systems-with-python/9781785882951.

[49] S. Keele et al. 2007. *Guidelines for performing systematic literature reviews in software engineering*. Technical Report. Technical report, Ver. 2.3 EBSE Technical Report. EBSE.

[50] J. Kephart and D. Chess. 2003. The vision of autonomic computing. *Computer* 1 (2003), 41–50.

[51] M. A. Khan and H. Tembine. 2017. Meta-learning for realizing self-x management of future networks. *IEEE Access* 5 (2017), 19072–19083. https://doi.org/10.1109/ACCESS.2017.2745999

[52] P. V. Klaine, M. A. Imran, O. Onireti, and R. D. Souza. 2017. A survey of machine learning techniques applied to self-organizing cellular networks. *IEEE Commun. Surveys Tutor.* 19, 4 (2017), 2392–2431. https://doi.org/10.1109/COMST.2017.2727878

[53] D. Kramer and W. Karl. 2012. Realizing a proactive, self-optimizing system behavior within adaptive, heterogeneous many-core architectures. In *Proceedings of the IEEE 6th International Conference on Self-Adaptive and Self-Organizing Systems*. 39–48. https://doi.org/10.1109/SASO.2012.26

[54] J. Kramer and J. Magee. 2007. Self-managed systems: An architectural challenge. In *Proceedings of the Conference on the Future of Software Engineering (FOSE'07)*. 259–268.

[55] C. Krupitzer, J. Otto, F. M. Roth, A. Frommgen, and C. Becker. 2017. Adding self-improvement to an autonomic traffic management system. In *Proceedings of the IEEE International Conference on Autonomic Computing (ICAC'17)*. 209–214. https://doi.org/10.1109/ICAC.2017.16

[56] C. Krupitzer, M. Pfannemuller, J. Kaddour, and C. Becker. 2018. SATISFy: Towards a self-learning analyzer for time series forecasting in self-improving systems. In *Proceedings of the IEEE International Workshops on Foundations and Applications of Self Systems*. 182–189. https://doi.org/10.1109/FAS-W.2018.00045

[57] A. Kurakin, I. Goodfellow, and S. Bengio. 2017. Adversarial machine learning at scale. Retrieved from https://arxiv.org/abs/1611.01236.

[58] E. Lee, Y.-D. Seo, and Y.-G. Kim. 2019. A Nash equilibrium based decision-making method for internet of things. *J. Ambient Intell. Human. Comput.* (2019), 1–9. https://doi.org/10.1007/s12652-019-01367-2

[59] T. Liu, C. Yang, C. Hu, H. Wang, L. Li, D. Cao, and F. Wang. 2018. Reinforcement learning-based predictive control for autonomous electrified vehicles. In *Proceedings of the IEEE Intelligent Vehicles Symposium (IV'18)*. 185–190. https://doi.org/10.1109/IVS.2018.8500719

[60] Y. Liu, D. Bai, and W. Jiao. 2018. Generating adaptation rules of software systems: A method based on genetic algorithm. In *Proceedings of the 10th International Conference on Machine Learning and Computing*. https://doi.org/10.1145/3195106.3195137

[61] T. Lorido-Botran, J. Miguel-Alonso, and J. Lozano. 2014. A review of auto-scaling techniques for elastic applications in cloud environments. *J. Grid Comput.* 12, 4 (2014), 559–592.

[62] M. Maggio, H. Hoffmann, A. Papadopoulos, J. Panerati, M. Santambrogio, A. Agarwal, and A. Leva. 2012. Comparison of decision-making strategies for self-optimization in autonomic computing systems. *ACM Trans. Auton. Adapt. Syst.* 7, 4 (2012). https://doi.org/10.1145/2382570.2382572

[63] S. Mahdavi-Hezavehi, V. Durelli, D. Weyns, and P. Avgeriou. 2017. A systematic literature review on methods that handle multiple quality attributes in architecture-based self-adaptive systems. *Info. Softw. Technol.* 90 (2017), 1–26. https://doi.org/10.1016/j.infsof.2017.03.013

[64] M. Masdari and A. Khoshnevis. 2019. A survey and classification of the workload forecasting methods in cloud computing. *Cluster Comput.* 23 (2019), 1–26.

[65] T. M. Mitchell. 1997. *Machine Learning*. McGraw-Hill. Retrieved from https://books.google.be/books?id=EoYBngEACAAJ.

[66] M. Moghadam, M.d Saadatmand, M.s Borg, M. Bohlin, and B. Lisper. 2018. Adaptive runtime response time control in plc-based real-time systems using reinforcement learning. In *Proceedings of the 13th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. https://doi.org/10.1145/3194133.3194153

[67] A. Pandey, B. Schmerl, and D. Garlan. 2017. Instance-based learning for hybrid planning. In *Proceedings of the IEEE 2nd International Workshops on Foundations and Applications of Self* Systems (FAS*W'17)*. 64–69. https://doi.org/10.1109/FAS-W.2017.122

[68] D. Papamartzivanos, F. Gomez Marmol, and G. Kambourakis. 2019. Introducing deep learning self-adaptive misuse network intrusion detection systems. *IEEE Access* 7 (2019), 13546–13560. https://doi.org/10.1109/ACCESS.2019.2893871

[69] Van Dyke Parunak and S. Brueckner. 2015. Software engineering for self-organizing systems. *Knowl. Eng. Rev.* 30, 4 (2015), 419–434. https://doi.org/10.1017/S0269888915000089

[70] A. Pelaez, A. Quiroz, and M. Parashar. 2016. Dynamic adaptation of policies using machine learning. In *Proceedings of the International Symposium on Cluster, Cloud and Grid Computing (CCGrid'16)*. IEEE, 501–510.

[71] L. Prechelt, D. Graziotin, and D. Mendez Fernandez. 2018. A community's perspective on the status and future of peer review in software engineering. *Info. Softw. Technol.* 95 (2018), 75–85.

[72] W. Qian, X. Peng, B. Chen, J. Mylopoulos, H. Wang, and W. Zhao. 2015. Rationalism with a dose of empiricism: Combining goal reasoning and case-based reasoning for self-adaptive software systems. *Require. Eng.* 20, 3 (2015), 233–252.

[73] X. Qin, W. Wang, W. Zhang, J. Wei, X. Zhao, and T. Huang. 2012. Elasticat: A load rebalancing framework for cloud-based key-value stores. In *Proceedings of the 19th International Conference on High Performance Computing*. 1–10.

[74] X. Qin, W. Wang, W. Zhang, J. Wei, X. Zhao, H. Zhong, and T. Huang. 2014. PRESC^2: Efficient self-reconfiguration of cache strategies for elastic caching platforms. *Computing* 96, 5 (2014), 415–451.

[75] F. Quin, D. Weyns, T. Bamelis, S. Buttar, and S. Michiels. 2019. Efficient analysis of large adaptation spaces in self-adaptive systems using machine learning. In *Proceedings of the 14th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. IEEE Press. https://doi.org/10.1109/SEAMS.2019.00011

[76] G. S. Blair, N. Bencomo, and R. France. 2009. Models@ run.time. *Computer* 42 (11 2009), 22–27. https://doi.org/10.1109/MC.2009.326

[77] F. Salfner and M. Malek. 2010. *Architecting Dependable Systems with Proactive Fault Management*. Springer. https://doi.org/10.1007/978-3-642-17245-8_8

[78] T. R. D. Saputri and S. W. Lee. 2020. The application of machine learning in self-adaptive systems: A systematic literature review. *IEEE Access* 8 (2020), 205948–205967. https://doi.org/10.1109/ACCESS.2020.3036037

[79] Burr Settles. 2009. *Active learning literature survey*. Technical Report. University of Wisconsin-Madison Department of Computer Sciences.

[80] S. Shalev-Shwartz and S. Ben-David. 2014. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press.

[81] A. M. Sharifloo, A. Metzger, C. Quinton, L. Baresi, and K. Pohl. 2016. Learning and evolution in dynamic software product lines. In *Proceedings of the IEEE/ACM 11th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS'16)*. 158–164. https://doi.org/10.1109/SEAMS.2016.026

[82] S. Sheikhi and S. Babamir. 2018. Using a recurrent artificial neural network for dynamic self-adaptation of cluster-based web-server systems. *Appl. Intell.* 48, 8 (2018), 2097–2111.

[83] S. Shevtsov, M. Berekmeri, D. Weyns, and M. Maggio. 2018. Control-theoretical software adaptation: A systematic literature review. *IEEE Trans. Softw. Eng.* 44, 8 (Aug. 2018), 784–810. https://doi.org/10.1109/TSE.2017.2704579

[84] K. Skałkowski and K. Zieliński. 2013. Automatic adaptation of SOA systems supported by machine learning. In *Technological Innovation for the Internet of Things*. Springer.

[85] M. Sommer, S. Tomforde, J. Hahner, and D. Auer. 2015. Learning a dynamic re-combination strategy of forecast techniques at runtime. In *Proceedings of the IEEE International Conference on Autonomic Computing*. 261–266. https://doi.org/10.1109/ICAC.2015.70

[86] A. Stein, S. Tomforde, A. Diaconescu, J. Hahner, and C. Muller-Schloer. 2018. A concept for proactive knowledge construction in self-learning autonomous systems. In *Proceedings of the IEEE 3rd International Workshops on Foundations and Applications of Self* Systems (FAS*W'18)*. 204–213. https://doi.org/10.1109/FAS-W.2018.00048

[87] A. Strauss and J. Corbin. 1990. *Basics of Qualitative Research: Grounded Theory Procedures and Techniques*. SAGE.

[88] D. Sykes, D. Corapi, J. Magee, J. Kramer, A. Russo, and K. Inoue. 2013. Learning revised models for planning in adaptive systems. In *Proceedings of the International Conference on Software Engineering*. IEEE Press.

[89] Z. Tang, W. Wang, L. Sun, Y. Huang, H. Wu, J. Wei, and T. Huang. 2018. IO dependent SSD cache allocation for elastic Hadoop applications. *Sci. China Info. Sci.* 61, 5 (2018), 050104.

[90] G. Tesauro, N. Jong, R. Das, and M. Bennani. 2007. On the use of hybrid reinforcement learning for autonomic resource allocation. *Cluster Comput.* 10, 3 (2007), 287–299.

[91] S. Thrun and T. Mitchell. 1995. Lifelong robot learning. *Robot. Auton. Syst.* 15, 1–2 (1995), 25–46.

[92] R. Van Solingen, V. Basili, G. Caldiera, and D. Rombach. 2002. Goal question metric (GQM) approach. *Encyclopedia of Software Engineering*. CRC Press, Boca Raton, FL.

[93] N. Villegas, H. Müller, G. Tamura, L. Duchien, and R. Casallas. 2011. A framework for evaluating quality-driven self-adaptive software systems. In *Proceedings of the 6th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. Association for Computing Machinery, New York, NY, 80–89. https://doi.org/10.1145/1988008.1988020

[94] M. Vollstedt and S. Rezat. 2019. *An Introduction to Grounded Theory with a Special Focus on Axial Coding and the Coding Paradigm*. Springer. https://doi.org/10.1007/978-3-030-15636-7_4

[95] J. Wan, Q. Li, L. Wang, L. He, and Y. Li. 2017. A self-adaptation framework for dealing with the complexities of software changes. In *Proceedings of the 8th IEEE International Conference on Software Engineering and Service Science (ICSESS'17)*. 521–524. https://doi.org/10.1109/ICSESS.2017.8342969

[96] D. Weyns. 2019. Software engineering of self-adaptive systems. In *Handbook of Software Engineering*. Springer International Publishing, Cham, 399–443. https://doi.org/10.1007/978-3-030-00262-6_11

[97] D. Weyns. 2020. *Introduction to Self-adaptive Systems: A Contemporary Software Engineering Perspective*. Wiley.

[98] D. Weyns and T. Ahmad. 2013. Claims and evidence for architecture-based self-adaptation: a systematic literature review. In *Proceedings of the European Conference on Software Architecture*. Springer, 249–265.

[99] D. Weyns and U. Iftikhar. 2016. Model-based simulation at runtime for self-adaptive systems. In *Proceedings of the IEEE International Conference on Autonomic Computing (ICAC'16)*. 364–373. https://doi.org/10.1109/ICAC.2016.67

[100] D. Weyns, U. Iftikhar, D. Hughes, and N. Matthys. 2018. Applying architecture-based adaptation to automate the management of internet-of-things. In *Software Architecture*, C. Cuesta, D. Garlan, and J. Pérez (Eds.). Springer, 49–67.

[101] D. Weyns, U. Iftikhar, D. Hughes, and N. Matthys. 2018. Applying architecture-based adaptation to automate the management of internet-of-things. In *Software Architecture*, C. Cuesta, D. Garlan, and J. Pérez (Eds.). Springer, 49–67.

[102] D. Weyns, S. Malek, and J. Andersson. 2012. FORMS: Unifying reference model for formal specification of distributed self-adaptive systems. *ACM Trans. Auton. Adapt. Syst.* 7, 1 (2012), 8.

[103] Bin Xiao. 2017. Self-evolvable knowledge-enhanced IoT data mobility for smart environment. In *Proceedings of the 1st International Conference on Internet of Things and Machine Learning*. 1–14.

[104] Satoru Yamagata, Hiroyuki Nakagawa, Yuichi Sei, Yasuyuki Tahara, and Akihiko Ohsuga. 2019. Self-adaptation for heterogeneous client-server online games. In *Proceedings of the International Conference on Intelligence Science*. Springer, 65–79.

[105] C. Zannier, G. Melnik, and F. Maurer. 2006. On the success of empirical studies in the international conference on software engineering. In *Proceedings of the 28th International Conference on Software Engineering*. https://doi.org/10.1145/1134285.1134333

[106] S. Žapčević and P. Butala. 2013. Adaptive process control based on a self-learning mechanism in autonomous manufacturing systems. *Int. J. Adv. Manufactur. Technol.* 66, 9–12 (2013), 1725–1743.

[107] T. Zhao, W. Zhang, H. Zhao, and Z. Jin. 2017. A reinforcement learning-based framework for the generation and evolution of adaptation rules. In *Proceedings of the IEEE International Conference on Autonomic Computing (ICAC'17)*. 103–112. https://doi.org/10.1109/ICAC.2017.47

[108] X. Zuo, G. Zhang, and W. Tan. 2014. Self-adaptive learning pso-based deadline constrained task scheduling for hybrid IAAS cloud. *IEEE Trans. Autom. Sci. Eng.* 11, 2 (2014), 564–573. https://doi.org/10.1109/TASE.2013.2272758

Received October 2020; revised February 2021; accepted June 2021